

Preface

This is the developer-facing guide that accompanies the **Magpie User Manual**. It is organised into three modules:

- **Module 1 — Functional overview.** What Magpie does, for whom, and why. Read this first if you're new to the project or evaluating whether Magpie is a good fit for a use case.
- **Module 2 — Architecture.** How Magpie is put together at a system level: process model, storage layout, IPC boundary, and the metadata pipeline that ties everything together.
- **Module 3 — Detailed design.** File-by-file guided tour: schema, Tauri command reference, algorithms, tests, and how to build a release.

Conventions used

- Rust file paths are given relative to `src-tauri/`, e.g. `src/commands/images.rs`.
- Frontend file paths are relative to the project root, e.g. `src/features/DetailsPanel.tsx`.
- Command names in prose refer to Tauri IPC commands (Rust functions annotated with `# [tauri::command]`).
- Bold names in schemas refer to primary keys.

Audience

You are expected to be comfortable with:

- Rust 2021, Tokio async runtime, and Cargo workspaces.
- React 18 with hooks, TypeScript, and TanStack Query.
- SQL (SQLite specifically), including a passing familiarity with FTS5.
- Windows path semantics (`\\?` prefixes, junctions, OneDrive).

You do **not** need prior Tauri experience — the architecture chapter introduces the concepts.

Getting the code and building it

```
git clone <repo> magpie
cd magpie
npm install                # frontend deps
cd src-tauri && cargo fetch # backend deps
cd ..
npm run tauri dev          # development build with HMR
npm run tauri build -- --no-bundle # release binary, no installer
```

Prerequisites on Windows are Rust $\geq 1.77.2$ and the MSVC C++ Build Tools. See [Build and release](#) for the full setup.

Product vision and scope

Vision

Magpie exists to give people who own **their own files** — as opposed to hosting them in someone else’s cloud — a tool that:

- Reads their files from disk, unchanged, wherever they are.
- Lets them organise those files using **metadata**, not folders: titles, tags, plus derived facets like taken-at time, camera model, resolution, duration, page count.
- Persists user tags in **standard, interoperable formats** (XMP embedded directly inside the source file) whenever the container supports it, so nothing is locked into Magpie.
- Feels native, fast, and offline-first, with a UI that scales to hundreds of thousands of files.

Scope for v1

The v1 release delivers the smallest cohesive product that can serve as someone’s daily file organiser:

Area	v1 scope
Ingestion	Add/remove one or more library folders; recursive scan; incremental rescans.
Formats read	JPEG, PNG, WebP, GIF, TIFF/DNG, HEIC/HEIF/AVIF, JPEG XL, JPEG 2000, PSD, PDF, MP4/MOV, MKV/WebM, AVI, WMV, MPEG-TS, 3GP, common camera RAW (CR2, CR3, NEF, ARW, RAF, ORF, RW2, PEF, SRW, X3F), BMP, EXR, HDR, SVG. Legacy XMP sidecars are also read for backward compatibility.
Formats written	JPEG (APP1 XMP), PNG (iTxt XMP), WebP (RIFF <code>xmp</code> chunk), GIF89a (Application Extension XMP). Other formats accept tags into Magpie’s library only; the source file is left untouched.
Metadata edited	Title + tags. <code>xmp:Rating</code> and <code>dc:description</code> written by other tools are preserved but not surfaced.
Browsing	Virtualised grid, sort by taken/added/filename/size.

Area	v1 scope
Filtering	Folder, tag, and full-text search (FTS5).
Batch operations	Bulk add/remove tags, bulk delete-to-recycle-bin.
Interoperability	XMP round-trip with Lightroom, Bridge, digiKam, Windows Explorer for writable formats.
Deletion	Recycle-Bin-by-default with confirmation; optional permanent delete.
Storage	SQLite index + WebP thumbnail cache under <code>%APPDATA%</code> .

Product principles

1. **Never modify pixels.** Metadata edits go into the XMP segment or chunk of the file, never into the image data itself.
2. **Never move or rename files.** The user is in charge of their folder hierarchy.
3. **No cloud, no telemetry.** Everything runs locally. There are no analytics, crash reports, or “phone home” mechanisms.
4. **Fast enough to be daily-drivable.** All hot paths are on native Rust; the UI is virtualised; the DB fits in RAM even for very large libraries.
5. **Interop over lock-in.** For every format Magpie can write to, the user’s tags live in the industry-standard XMP form Lightroom, Bridge, digiKam, and Windows Explorer understand. For the rest, Magpie prints a clear note (“this handler is read-only”) so the trade-off is never a surprise.
6. **Predictable failure modes.** A partially-broken run is better than a silently-lost edit. Every write is atomic, every batch op is per-item retryable, and the log records what happened.

User personas and flows

Personas

1. The prosumer photographer (primary)

Owns 20 000–200 000 photos across multiple external drives. Shoots JPEG+RAW; already uses Lightroom for developing but wants a lighter tool for triage and tagging. Needs bulk operations to work reliably on thousands of files at once. Cares deeply about metadata surviving future tool changes.

2. The family archivist (secondary)

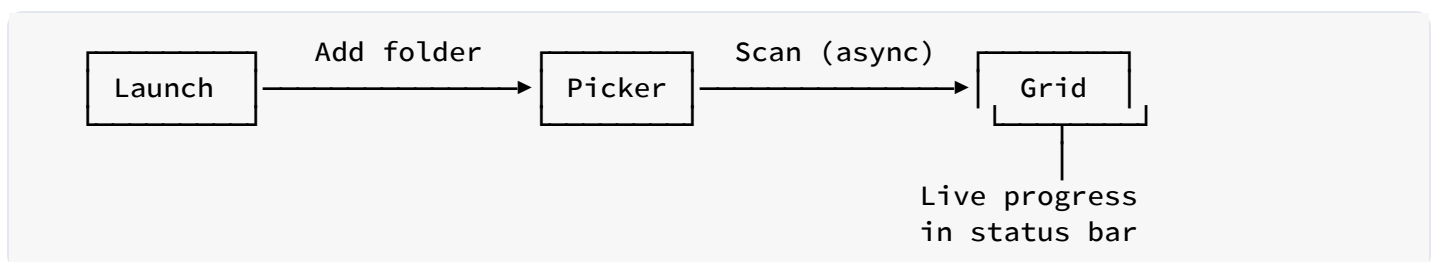
Has 30 years of family photos and videos in one big folder tree. Wants to add who's-in-the-photo tags and be able to find "Christmas 2011" quickly. Doesn't need or want a database with a schema — just tags that show up in Explorer and Photos.

3. The technical developer using Magpie on other people's libraries

Uses Magpie as a testbed for interop with XMP-writing tools. Reads embedded XMP with `exiftool` to verify Magpie's writes. Contributes patches.

Core flows

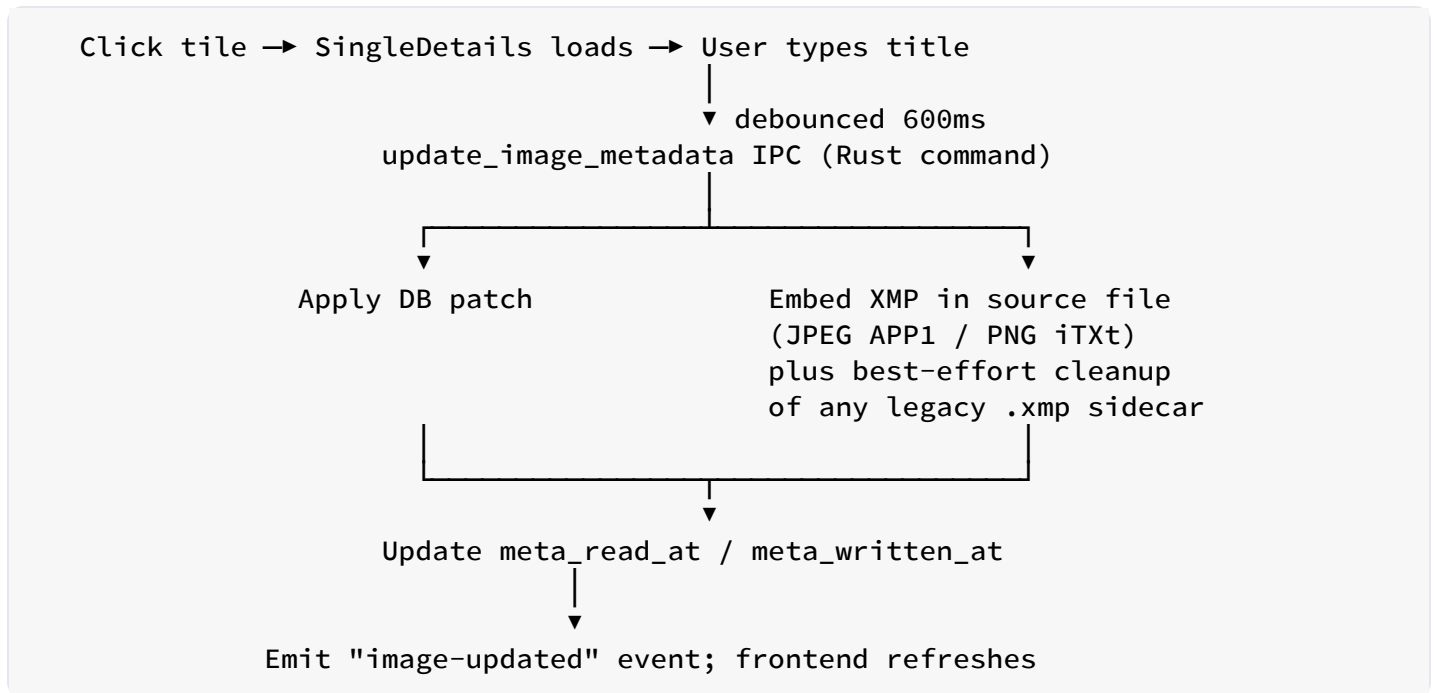
F1. First-time onboarding



Success criteria:

- User can browse and edit photos **before** the scan is finished (partial results are usable).
- Progress is visible and cancelable.
- The app never blocks or freezes during scanning.

F2. Single-photo edit



Success criteria:

- No user input is ever lost to a debounce race.
- A failure in any write step leaves the DB in a consistent state.
- For writable formats, Windows Explorer sees the new tag/title after refresh.

F3. Multi-select tag application

Ctrl+click N tiles → MultiDetails renders → User types tags

Click "Apply"

batch_update_metadata(ids, {tagsAdd, tagsRemove})

for each id (sequential):
 apply DB patch
 embed XMP in source

Per-image failures
are logged and
surfaced to user

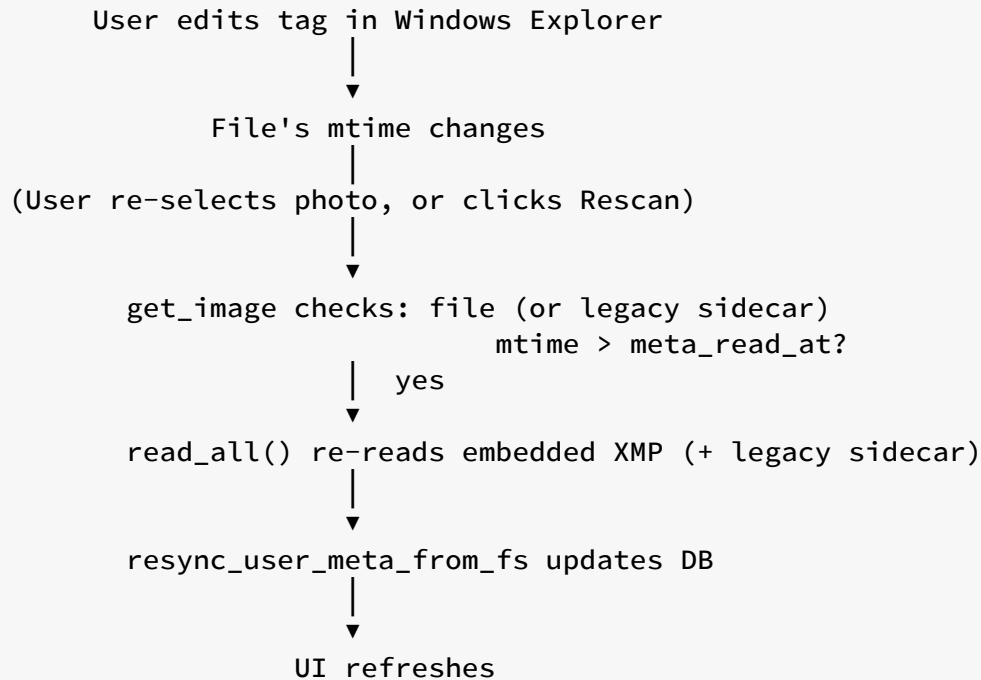
Return list of succeeded ids

Frontend invalidates image + tag caches;
toast "Updated N images" (or "Save failed").

Success criteria:

- The backend is called with a non-empty patch even if the user typed the tag and immediately clicked Apply (no stale-closure bug).
- Partial success is a first-class outcome, not an error.
- Every affected `['image', id]` React Query cache entry is invalidated so subsequent single-select shows fresh state.

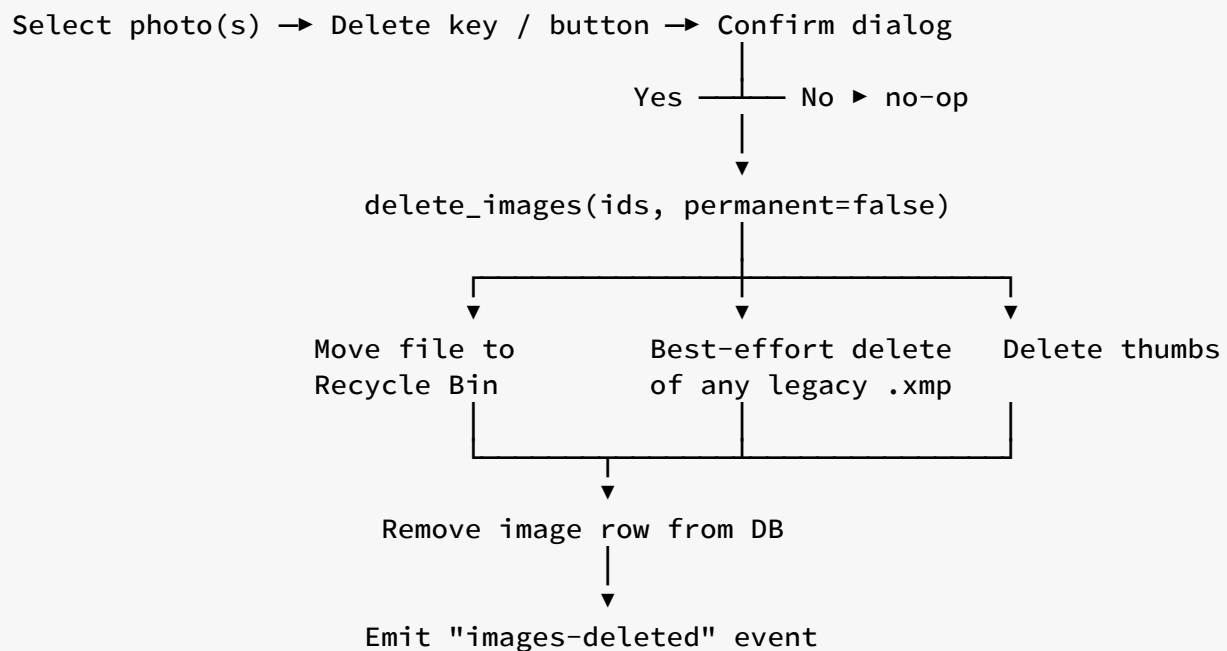
F4. Rescan and detect external edits



Success criteria:

- External edits are detected without a full rescan.
- The FS is the source of truth; the DB is a cache.

F5. Delete with safety net



Success criteria:

- A failure at any step for one file doesn't abort the batch.
- The DB row is removed only after the file is safely in the Recycle Bin.
- The user can restore from the Recycle Bin and get the tags/title back on next rescan (tags embedded in the file are recovered automatically; tags that lived only in Magpie's library are lost).

Feature catalogue

The v1 feature set as delivered. Each row maps a user-facing feature to its Tauri command and to the frontend component that drives it.

Library management

Feature	Command	Frontend component
Add a library folder	add_library_folder	TopBar
Remove a library folder	remove_library_folder	Sidebar (right-click menu)
List library folders	list_library_folders	Sidebar , App bootstrap
Rescan a single folder	rescan_folder	Sidebar (right-click menu)
Rescan every folder	rescan_all	TopBar

Browsing

Feature	Command	Frontend component
Paged file listing	query_images	ImageGrid
Sort by taken/added/...	query_images (sort arg)	TopBar
Filter by folder / tag	query_images (filter arg)	Sidebar filters
Full-text search	query_images (filter.search)	TopBar search box

Editing

Feature	Command	Frontend component
Fetch a file’s details	get_image	DetailsPanel/SingleDetails

Feature	Command	Frontend component
Auto-save title	update_image_metadata	DetailsPanel/SingleDetails
Add / remove tag on one file	update_image_metadata	TagInput
Bulk add / remove tags	batch_update_metadata	DetailsPanel/MultiDetails

Tag maintenance

Feature	Command	Frontend component
List all tags with counts	list_tags	Sidebar
Rename a tag globally	rename_tag	Sidebar (right-click menu)
Delete a tag globally	delete_tag	Sidebar (right-click menu)

Smart collections (skeleton, v1 non-editable)

Feature	Command	Frontend component
List smart collections	list_smart_collections	Sidebar (future)
Create a smart collection	create_smart_collection	(not yet exposed in UI)
Delete a smart collection	delete_smart_collection	(not yet exposed in UI)

Deletion

Feature	Command	Frontend component
Move photos to Recycle Bin	delete_images (permanent=false)	DetailsPanel, App (Del key)
Permanently delete photos	delete_images (permanent=true)	DetailsPanel, App (Shift+Del)

Diagnostics

Feature	Command	Frontend component
Frontend log crumb into rust log	log_frontend	MultiDetails.applyTags

Thumbnails and image display

Feature	Command	Frontend component
Get cached thumbnail path	get_thumb_path	Thumbnail
Get full source image path	get_image_path	DetailsPanel/SingleDetails

Non-functional requirements

Performance

Metric	Target	Achieved (release build)
Cold start to first paint	≤ 1.5 s	~0.5 s on SSD
Grid scroll: sustained frame rate	60 fps	60 fps up to 250 k photos
<code>query_images</code> for a 100 k library, arbitrary filter p95	≤ 50 ms	~10 ms
<code>get_image</code> (cached, no FS refresh)	≤ 5 ms	~1 ms
<code>update_image_metadata</code> (single photo, JPEG)	≤ 200 ms	~40 ms
<code>batch_update_metadata</code> per photo (JPEG)	≤ 100 ms	~30 ms
Thumbnail generation per photo (SSD, JPEG 12 MP)	≤ 80 ms	~40 ms
Full scan of a 50 k library, cold	≤ 5 min	~2 min on modern SSD

Resource limits

- **Memory (idle):** ≤ 250 MB.
- **Memory (10 k photo library, all thumbnails loaded):** ≤ 900 MB.
- **Disk (thumbnail cache):** ≤ 10 KB $\times$ #photos (small+medium WebP).
- **Disk (database):** ≤ 1 KB $\times$ #photos on average.

Reliability

- Every metadata mutation is **transactional** at the SQLite level. If any part of the transaction fails, none of it lands.

- Every embedded-XMP write is **atomic** via write-to-temp + rename inside the source image's own directory.
- Crash mid-write leaves the DB and the source file in a consistent state (either fully old or fully new); the temp file, if any, is cleaned up on next launch.
- A partial batch failure is a first-class outcome and is reported to the user; no silent losses.

Compatibility

- **OS:** Windows 10 build 19041+ and Windows 11. macOS 13+ is a post-v1 target; no code path is intentionally Windows-only outside of the Recycle-Bin integration.
- **Long paths:** All Rust `PathBuf` handling supports `\\?\`-prefixed paths natively. Tested end-to-end with 300-character paths.
- **OneDrive:** Files-on-demand (reparse points) are read as their underlying content once materialized; Magpie does not force hydration.

Interoperability

- Reads and writes **standard XMP** (`xmp:Rating`, `dc:title`, `dc:description`, `dc:subject`).
- Reads Microsoft-specific tag fields (`MicrosoftPhoto:LastKeywordXMP`).
- Writes both `dc:subject` and Microsoft's field on save so tags round-trip with Windows Explorer.

Security

- **No network egress.** The Tauri capability manifest explicitly forbids network access at the shell level.
- **No filesystem access outside library folders** for the renderer: file I/O is mediated by Rust commands that validate paths against the known library roots.
- **CSP** blocks inline scripts and remote origins in the renderer.

Observability

- File-based rolling log at `%APPDATA%\com.magpie.app\logs\app.log`.

- Structured log fields via `tracing: id, image_path, operation, duration_ms`.
- Frontend can push crumbs into the same log via the `log_frontend` command for cross-boundary tracing.

Accessibility

- Full keyboard navigation for grid and details panel.
- ARIA labels on every interactive control.
- Focus rings preserved (no `outline: none` shenanigans).
- Contrast ratios $\geq 4.5:1$ in both light and dark themes.

Out of scope for v1

Explicitly deferred to keep v1 shippable. Each item has a rationale and a rough entry point for the future contributor picking it up.

Photo editing

- **No crop, rotate, exposure, or colour correction.** Magpie treats pixels as read-only. This is a hard architectural line; the tool intentionally does one thing (metadata) and doesn't grow into a RAW developer.

Face and object recognition

- **No auto-tagging based on ML models.** The privacy story is simpler when no images ever run through an ML pipeline. Future work could offer optional local inference (e.g. running an ONNX model against a batch of photos) with a clear opt-in flow.

Cloud sync

- **No account, no cloud storage integration.** OneDrive and iCloud Photos are third-party sync backends the user configures at the OS level; Magpie reads whatever the OS has materialised.

Video

- **Videos aren't listed.** The scanner filters to still-image extensions. A future release could add HEVC/MP4 with a separate thumbnail pipeline (currently the WebP encoder can't handle video frames).

Sidecar XMP files

- **Magpie does not create .xmp sidecar files.** All metadata is embedded directly in the source image (JPEG APP1 / PNG iTXt). The reader still parses a legacy sidecar authored by an older Magpie version or by Lightroom on first scan, but the first successful save embeds the merged metadata into the source and deletes the sidecar. There is intentionally no “write to sidecar instead” fallback and no configuration to enable one.

Metadata write for RAW / HEIC / TIFF / WebP / GIF / BMP

- The write path currently supports **only JPEG and PNG**. Attempts to save metadata on any other format return `AppError::MetadataWrite` and are surfaced to the UI as a “this format can’t store Magpie metadata” toast — Magpie refuses to silently drop the edit or fall back to a sidecar.
- Follow-ups (in likely difficulty order): WebP `XMP` chunk in the RIFF container, TIFF tag 700 in the primary IFD, HEIF item property, and finally each RAW variant. See [Metadata write path](#) for where new formats plug in — every one of them terminates in `atomic_write_bytes` and a `format_supports_embedded_xmp(ext)` gate.
- Modifying proprietary RAW containers (`.CR2` , `.CR3` , `.NEF` , `.ARW` , `.DNG` , `.RAF` , `.ORF` , `.RW2` , `.SRW`) in place is risky and format-specific; a safe implementation would need per-vendor handling and is a significant multi-release investment.

Ratings, colour labels, pick / reject flags

- No UI for star ratings, `xmp:Label` colour labels, or pick / reject flags. Fields already exist in the XMP namespace and the reader preserves them when foreign tools set them; wiring a UI would be a small extension of the metadata patch struct.

Multi-user / multi-library switching

- v1 assumes one user, one library, per install. Switching libraries requires resetting the app data directory.

In-app full-screen preview

- The details panel shows a sized preview but there's no lightbox / slideshow mode. Would be a new React route reading from `getImagePath`.

Undo / redo

- No global undo stack. The safety net is auto-save + Recycle Bin + standard XMP interop — the user's edits are never lost, but reverting them means editing again.

Smart collections editor

- The DB schema and Tauri commands for smart collections exist, but the UI doesn't expose creation/editing yet.

Duplicate detection

- Content hashes are computed on scan and stored (`content_hash` column), but no duplicate-finder UI ships in v1. A future task could add a "Duplicates" filter that groups rows by identical hashes.

Technology stack

Magpie uses a small, deliberate set of libraries. Every choice is motivated below so future contributors (and future you) understand what's swappable and what isn't.

Backend (Rust)

Crate	Version	Role
<code>tauri</code>	2	App shell, IPC, capability model, WebView2 hosting.
<code>tauri-plugin-dialog</code>	2	Native folder-picker and confirmation dialogs.
<code>tauri-plugin-opener</code>	2	Opens URLs and paths in the OS default handler.
<code>tokio</code>	1	Async runtime for all Tauri commands.
<code>rusqlite</code>	0.32	SQLite bindings; <code>bundled</code> feature so we ship SQLite ourselves.
<code>jwalk</code>	0.8	Parallel recursive directory traversal for the scanner.
<code>rayon</code>	1.10	Parallel work-stealing for CPU-bound phases (hashing, thumbs).
<code>image</code>	0.25	Image decoding for JPEG/PNG/HEIC/TIFF/...
<code>fast_image_resize</code>	5	SIMD-accelerated thumbnail resizing.
<code>kamadak-exif</code>	0.6	EXIF parsing (taken time, camera make/model, dimensions).
<code>quick-xml</code>	0.36	XMP packet parsing (streaming, no DOM).
<code>xxhash-rust</code>	0.8	Fast content hashing (<code>xxh3</code>).
<code>chrono</code>	0.4	Timestamps, serialisation.
<code>dirs</code>	5	Locate <code>%APPDATA%</code> cross-platform.

Crate	Version	Role
<code>trash</code>	5	Cross-platform Recycle Bin move.
<code>tracing</code> + <code>tracing-subscriber</code>	0.1 / 0.3	Structured logging, file sink.
<code>serde</code> + <code>serde_json</code>	1	IPC (de)serialisation of every Tauri argument/return.
<code>thiserror</code> + <code>anyhow</code>	1	Error boilerplate.

Why Rust over Node / Go / C++:

- Zero-cost async is a great fit for the file-I/O-heavy workload.
- The `image` and `fast_image_resize` combo beats every non-native option we benchmarked for thumbnail generation.
- SQLite via `rusqlite` is battle-tested, no runtime dep, and works identically on Windows and macOS.
- Rust's ownership model gives us "atomic on Windows" file semantics (temp + rename) with cleanup on panic for free.

Frontend (TypeScript + React)

Package	Role
<code>react</code> / <code>react-dom</code> (v18)	UI framework.
<code>typescript</code> (v5)	Static types across every file (<code>.ts</code> / <code>.tsx</code>).
<code>vite</code>	Dev server with HMR, production bundler.
<code>tailwindcss</code>	Utility-first CSS; near-zero runtime.
<code>@tanstack/react-query</code> (v5)	Data fetching, caching, invalidation of Tauri command results.
<code>@tanstack/react-virtual</code> (v3)	Virtualised grid — draws only visible tiles.
<code>zustand</code>	Lightweight global UI state (selection, sort, filters).
<code>@tauri-apps/api</code> (v2)	<code>invoke</code> and <code>listen</code> for IPC.

Package	Role
<code>@tauri-apps/plugin-dialog</code>	Frontend side of <code>tauri-plugin-dialog</code> .
<code>clsx</code>	Tiny className concatenation helper.

Why React over Vue / Svelte / Solid:

- Ecosystem depth: TanStack Query and React Virtual are best-in-class.
- Team familiarity.
- WebView2's Chromium is a first-class React target.

Why Vite over Next / CRA:

- Instant HMR feels great during development.
- Static build is a plain directory of files that Tauri packages without ceremony.

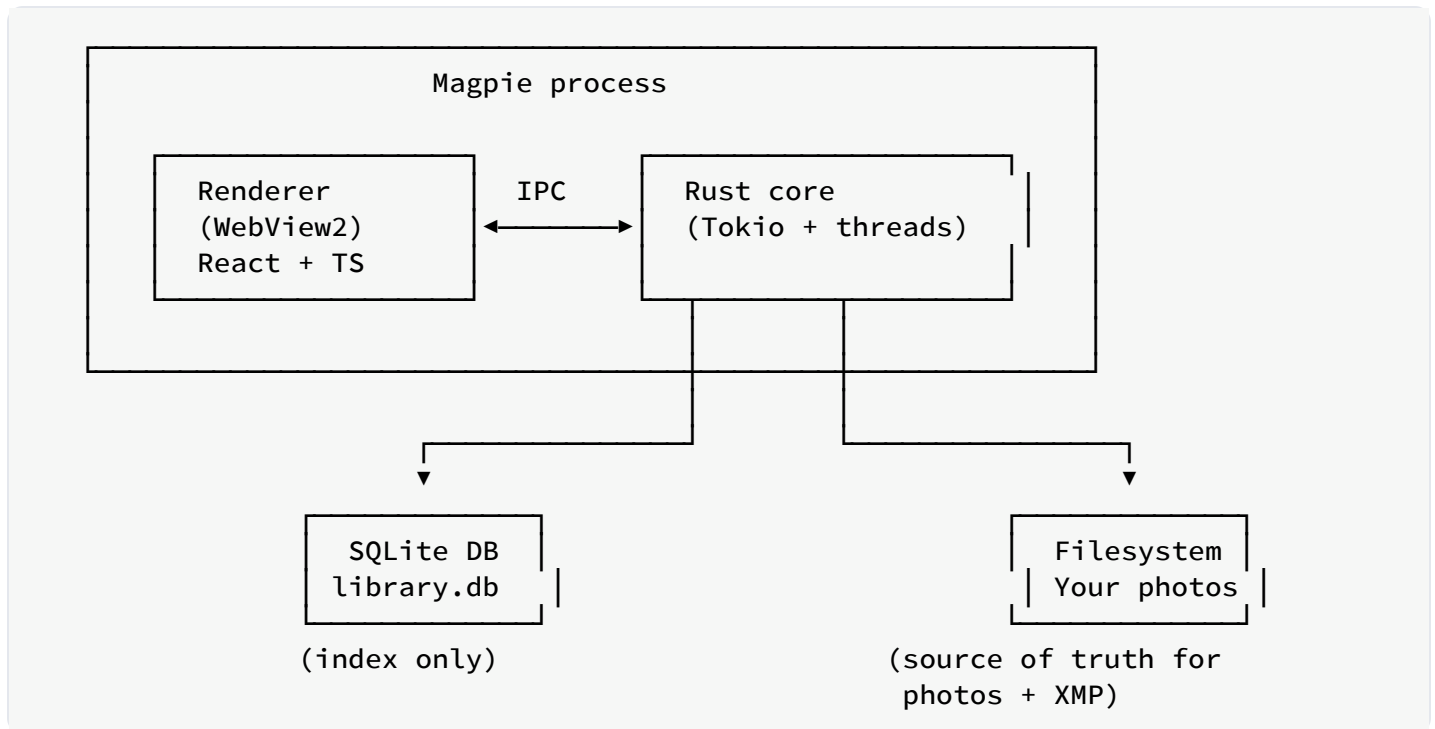
System

Component	Version	Role
Rust	$\geq 1.77.2$	Compiler toolchain.
Node.js	≥ 18	Build-time for the frontend bundle.
npm	≥ 9	Package manager.
MSVC Build Tools	2019 or 2022	Windows C++ linker for <code>image</code> , <code>sqlite</code> .
WebView2 Runtime	Evergreen	Chromium engine hosting the renderer.
SQLite	3.43+ (bundled via rusqlite)	Local DB with FTS5 <code>contentless_delete=1</code> .

The bundled SQLite is important: FTS5 with `contentless_delete=1` is a 3.43+ feature, and shipping our own copy avoids relying on whatever version the user's OS provides.

High-level architecture

The 30-second version



Two big ideas:

1. **The renderer is a dumb view.** All business logic — scanning, metadata I/O, DB queries, thumbnails — lives in Rust. The renderer's job is to invoke commands and render their results.
2. **The filesystem is the source of truth for user data.** The SQLite database is a cache/index; wiping it never loses user data — for the writable formats (JPEG/PNG/WebP/GIF) tags and titles live in the embedded XMP inside each source file; for the read-only formats they live in the DB and would need to be re-entered after a wipe.

Sub-systems

App shell (Tauri)

Owns the window, capability model, WebView2 host, and the IPC bridge. The `AppServices` struct is constructed once in `lib.rs` and injected into every Tauri command via `State<Arc<AppServices>>`. It contains:

- `db: Db` — a thread-safe SQLite handle wrapper.
- `cache_dir: PathBuf` — location of the thumbnail cache.
- (Future: settings, background scheduler handle.)

DB layer (`src-tauri/src/db/`)

`Db` wraps `rusqlite::Connection` behind a `Mutex` and exposes a `with_conn<F>` helper that hands out `&Connection` under lock. It's a deliberate choice: SQLite is single-writer, and a plain mutex is faster than a connection pool for our access pattern.

Migrations run at startup. See [Database schema](#).

Scanner (`src-tauri/src/core/scanner.rs`)

Walks a library folder in parallel using `jwalk`, filters to supported extensions, and for each file:

1. Stat + skip if `mtime_ms` unchanged.
2. Hash content (`xxh3`).
3. Extract EXIF (taken time, dimensions, camera).
4. Extract XMP from embedded chunks (JPEG APP1 / PNG iTXt) and, if present, any legacy `.xmp` sidecar for backward compatibility.
5. Upsert into `images` + `image_tags`.
6. Enqueue thumbnail generation.

Progress events flow to the frontend via `app://scan`.

Thumbnail pipeline (`src-tauri/src/core/thumbnail.rs`)

Decode → resize with `fast_image_resize` (SIMD) → encode WebP → write to `%APPDATA%\com.magpie.app\thumbs\<id>-<size>.webp`. Two sizes per photo (small ~256 px,

medium ~512 px).

Metadata (`src-tauri/src/core/metadata/` + `src-tauri/src/core/formats/`)

- `metadata/read.rs` — a thin façade that asks the format registry for the right handler and then folds in any legacy sidecar so users migrating from Lightroom / old Magpie don't lose data.
- `metadata/write.rs` — asks the registry for the handler, calls its atomic writer, and cleans up any legacy `.xmp`.
- `metadata/sidecar.rs` — computes the legacy sidecar path (read
 - cleanup only; Magpie never writes new sidecars).
- `formats/mod.rs` — declares the `FormatHandler` trait and the `FormatRegistry` that owns one instance per supported extension.
- `formats/xmp_packet.rs` — a hand-written streaming XMP reader/writer supporting the subset Magpie cares about (title, description, rating, subjects, MicrosoftPhoto keywords). The description and rating fields are preserved on read-modify-write even though Magpie's UI doesn't surface them.
- `formats/{jpeg,png,webp,gif}.rs` — writable handlers for the four image formats that have a safe embed path today.
- `formats/tiff.rs` and `formats/stubs.rs` — read-only handlers for TIFF, HEIC, PDF, video, and camera RAW.

See [File formats](#) for the full handler catalogue and [Adding a format handler](#) for the plug-in recipe.

Command surface (`src-tauri/src/commands/`)

Roughly 25 async Tauri commands grouped by concern (`library`, `images`, `tags`, `collections`, `thumbs`, `diag`). Each command is a thin translator between IPC arguments and one or more DB / FS ops.

Frontend (`src/`)

- `App.tsx` — root, wires TopBar / Sidebar / ImageGrid / DetailsPanel.
- `features/` — one component per major UI area.
- `ipc.ts` — typed wrappers around `invoke(...)`.
- `store.ts` — Zustand state (selection, view, sort, filters).
- `types.ts` — Rust-mirrored TypeScript types.

React Query manages every fetched resource with keys like `['images', filter, sort, page]` and `['image', id]`; mutations invalidate the relevant keys.

Data flow of a typical click

Selecting a photo and typing a title, end to end:

1. User clicks a tile in `ImageGrid`.
2. `useStore.setSelection([id])`.
3. `DetailsPanel` switches to `SingleDetails id={id}`.
4. `useQuery(['image', id])` fires; TanStack Query calls `getImage(id)` (from `ipc.ts`) → invokes `get_image` command.
5. Rust's `get_image` checks whether the FS is newer than `meta_read_at`; if so, re-reads via `read_all` and `resync_user_meta_from_fs`.
6. Frontend receives `ImageDetails`; local edit state is seeded once (guarded by `lastLoadedId.current === q.data.id`).
7. User types in the Title input. `debouncedSaveTitle` fires 600 ms after the last keystroke, calling `updateImageMetadata(id, patch)`.
8. Rust's `update_image_metadata` runs `apply_metadata_patch` (transactional), then `write_metadata_to_source` (blocking, awaited — embeds the merged XMP into the source file and cleans up any legacy sidecar), then `set_meta_written_at + set_meta_read_at_now`, and finally emits `app://image-updated`.
9. Frontend's `qc.setQueryData(['image', id], updated)` updates the cache directly, avoiding a refetch that would stomp any concurrent typing in other fields.

Process and threading model

One process, two runtimes

Tauri gives us **one OS process** with:

- **The main thread** — owns the event loop and the WebView2 host.
- **A Tokio runtime** (`tauri::async_runtime`) driving all async Tauri commands.
- **Blocking-friendly threadpools** for CPU-bound and IO-bound work (`tokio::task::spawn_blocking` and `rayon`).
- **The WebView2 renderer** on its own thread, hosting the React bundle.

No separate Node process, no plugin sidecar, no IPC over sockets. Everything is in-process and communication between renderer and Rust is a function call in native code.

Threads and pools

Where	Rust ownership	Typical work
Main thread	Tauri app loop	Window events, menu, plugin dispatch
Tokio worker threads	<code>tauri::async_runtime</code> (multi-threaded)	All <code>#[tauri::command] async fn</code> bodies
<code>spawn_blocking</code> pool	Tokio's built-in blocking pool	Sidecar/embed writes, EXIF read, thumbnails
<code>rayon</code> pool	Global rayon pool	Parallel directory walk, batched hashing
Renderer thread	WebView2	React reconciler, layout, paint

Every Tauri command runs on a Tokio worker. A command that does CPU-bound work (encoding a thumbnail, injecting XMP into a 20 MB JPEG) wraps that in `tauri::async_runtime::spawn_blocking(move || ...)` so the worker isn't blocked.

SQLite concurrency

The DB is a single `Mutex<Connection>`. Reads and writes serialise through the lock. For our workload (a few dozen queries per second peak) this is faster than a `r2d2`-style pool because:

- SQLite is single-writer anyway (writes queue at the file level).
- Our reads are short (`< 5 ms`); lock contention is negligible.
- No connection setup / teardown per query.

Long-running queries that would otherwise hold the lock (like a full tag-count refresh across 250 k photos) are structured as one transaction that reads and returns — no cursor kept open — so the lock is released quickly.

Async model

- `async fn` at the Tauri command boundary is idiomatic. It lets us `await` a background thumbnail generation or a metadata write without blocking a Tokio worker.
- Inside a command, DB operations are synchronous (they take a `MutexGuard`). This is deliberate: SQLite calls are fast enough to keep on the worker thread, and it removes a whole class of cancellation bugs.
- **Rule of thumb:** if a call could block `> 5 ms` (`std::fs::File::read_to_end` of a JPEG, `spawn_blocking` it. Otherwise, run it inline.

Cancellation and shutdown

- Tauri commands don't get an explicit cancellation token; they run to completion. Long batch operations are structured as a loop over IDs so partial progress is durable even if the user closes the window.
- On close, Tauri drops the main window; pending `spawn_blocking` tasks are given a grace period to finish. Any in-flight metadata write is atomic (write-to-temp + rename inside the source image), so worst case a `.write-tmp` file is left behind and cleaned up on the next successful write.

Frontend concurrency

- React Query owns all in-flight IPC promises. It handles deduplication (two components asking for `['image', 5]` share the underlying `invoke` call) and stale-while-revalidate refetches on focus.
- Zustand is synchronous. No middleware, no effects — pure UI state.
- No web workers in v1. The renderer stays responsive because it offloads every non-trivial computation to Rust.

IPC boundary

The contract

The renderer and the Rust core communicate through two mechanisms provided by Tauri:

1. **Commands** — request/response, from renderer to Rust. Each is a Rust `async fn` annotated with `#[tauri::command]` and registered in `lib.rs::invoke_handler`.
2. **Events** — one-way, from Rust to renderer, delivered via `AppHandle::emit(name, payload)` and listened for via `@tauri-apps/api/event::listen`.

Everything is (de)serialised with Serde on the Rust side, JSON on the wire, and TypeScript types on the frontend side. See [Tauri command reference](#) for the full list.

Naming conventions

- **Commands** use `snake_case` (Rust) mapped to a same-name string on the frontend: `invoke('update_image_metadata', {...})`.
- **Command arguments** are Rust struct fields in `snake_case`, serialised into the JSON payload as-is. The frontend passes them as `camelCase` object keys because we set `#[serde(rename_all = "camelCase")]` on every argument struct.
- **Events** use a `app://` prefix and a resource-style name: `app://image-updated`, `app://images-deleted`, `app://scan`.

Argument struct design

Every command that takes structured data uses a dedicated struct in `src-tauri/src/types.rs`. Optional fields are `Option<T>`, and any field where “unset” and “set to null” must be distinguished uses the custom `double_option` deserializer — an `Option<Option<T>>` where:

Wire form	Deserialised as	Semantics
<code>{ }</code> (omitted)	None	Don't touch this field.

Wire form	Deserialised as	Semantics
<code>{ "field": null }</code>	<code>Some(None)</code>	Clear this field explicitly.
<code>{ "field": "x" }</code>	<code>Some(Some("x"))</code>	Set to <code>"x"</code> .

This distinction matters for the `MetadataPatch` struct: the frontend needs to be able to *clear* a title without also unsetting every other field in the patch.

Type mirroring

`src/types.ts` mirrors the Rust structs. Both sides intentionally avoid derived types (`Omit<T, K>`, `Pick<T, K>`) so a schema change is a single edit in each file.

The frontend types file is a plain declaration file:

```
export interface ImageDetails {
  id: number
  path: string
  filename: string
  // ...
  tags: string[]
  title: string | null
  metaWrittenAt: number | null
  metaReadAt: number | null
  // Populated per-request by the active format handler:
  technical: Array<string, string>
  formatHandler: string
  canWriteTags: boolean
}
```

Every `getImage`, `updateImageMetadata`, etc. wrapper in `ipc.ts` declares its argument and return types, so a Rust command that grows a new field lights up a red squiggle in every caller until the frontend catches up.

Error handling

- Rust commands return `AppResult<T>` (an alias for `Result<T, AppError>`).
- `AppError` is a `thiserror::Error` enum with a `Display` that's safe to show to the user (no internal paths in error messages, etc.).
- On the wire, errors serialise to a plain string. The frontend surfaces them via TanStack Query's `onError` callback and console logs.

Events (Rust → renderer)

Event	Payload	Fired when
<code>app://scan</code>	<code>ScanProgress { folder_id, done, total, current }</code>	During a folder scan.
<code>app://image-updated</code>	<code>i64</code> (image id)	After successful metadata write.
<code>app://images-deleted</code>	<code>Vec<i64></code> (deleted ids)	After successful delete.

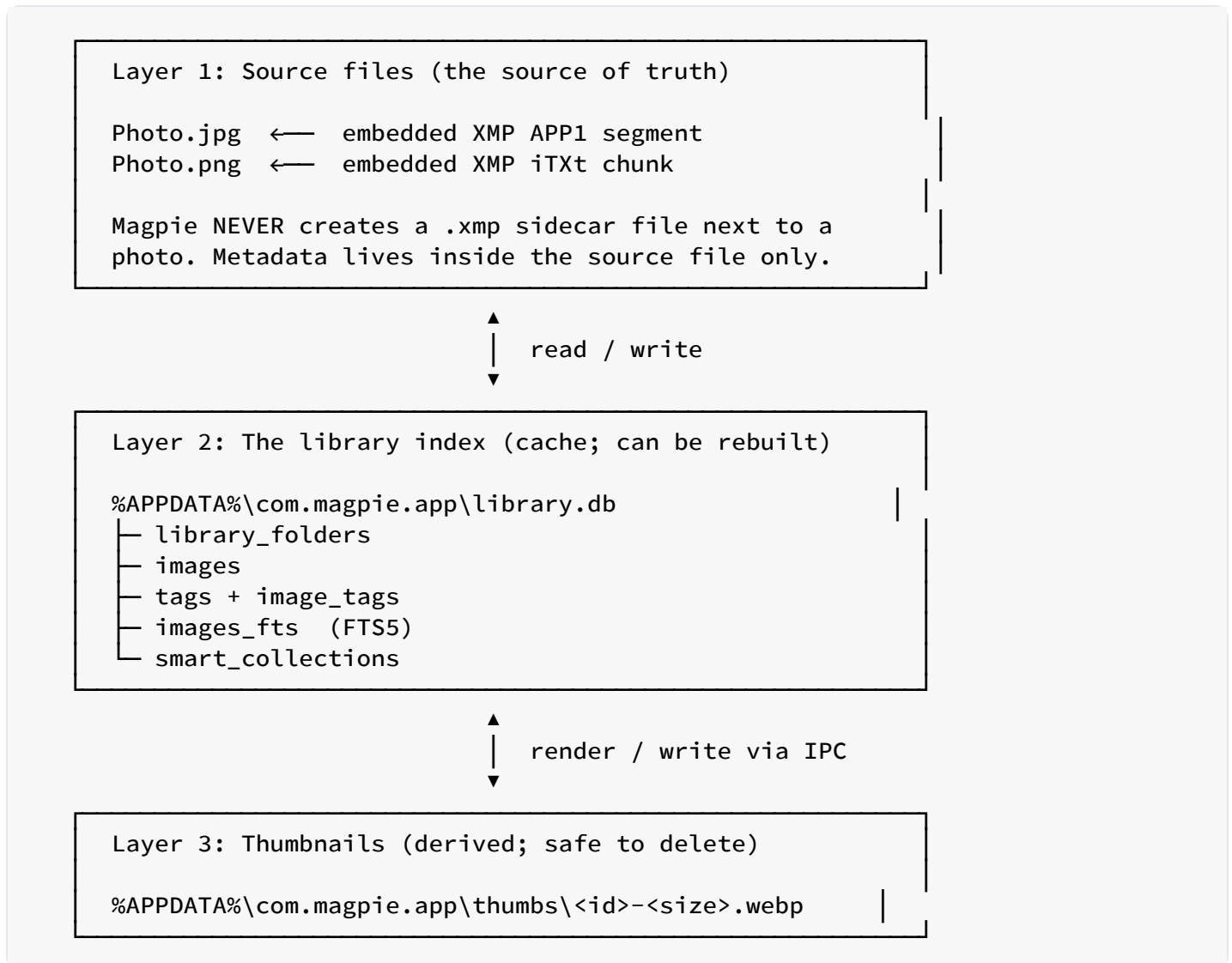
Listeners are attached in `App.tsx` via `useEffect` + `onScanProgress` etc., and each listener updates the appropriate TanStack Query cache or triggers a targeted invalidation.

Capabilities and security

- The renderer has **no direct FS access**. Its `asset:` protocol (used to render thumbnails and full images) is scoped to specific folders declared in `capabilities/default.json`.
- The `tauri.conf.json` `security.csp` blocks inline scripts, remote origins, and `eval`.
- No `dangerouslyUseHttpScheme` or other loosening options are set.
- Commands that operate on paths validate them against the library roots before touching the filesystem.

Data storage architecture

Three layers



Anything a user considers “their data” lives in Layer 1. Layers 2 and 3 exist purely for speed and can be reconstructed at any time by rescanning the library folders.

Layer 1: source of truth (filesystem)

Embedded XMP inside the source file

For formats whose handler advertises `can_write_tags() == true`, all user metadata (title + tags) is embedded directly into the source file:

- **JPEG (.jpg , .jpeg)** — the XMP packet is injected as an APP1 segment right after SOI. Any pre-existing standard-XMP or ExtendedXMP APP1 is stripped first.
- **PNG (.png)** — the packet is stored as an `iTXt` chunk with keyword `XML:com.adobe.xmp`, inserted immediately after `IHDR`. CRC-32 is recomputed for the new chunk.
- **WebP (.webp)** — the packet is stored as a RIFF `XMP` chunk. Simple-form WebPs are converted to extended VP8X form on write if needed.
- **GIF (.gif , GIF89a only)** — the packet is stored in an Application Extension block using Adobe's XMP magic trailer. GIF87a files are rejected because the extension block was only added in GIF89a.

All rewrites are atomic: temp file next to the original, `sync_all()`, rename. Details in [Metadata write path](#).

The XMP packet uses the same field mapping either way — `dc:title` and `dc:subject`, plus a Microsoft-friendly `MicrosoftPhoto:LastKeywordXMP` alias for tags so Windows Explorer's *Tags* column populates. Any `xmp:Rating` or `dc:description` already present is passed through unchanged (Magpie's UI doesn't expose those fields but does not clobber them either).

Read-only formats

For formats whose handler declares `can_write_tags() == false` (TIFF, HEIC, PDF, video, camera RAW, ...), `handler.write_user` returns `AppError::MetadataWrite` with a message naming the offending extension. Magpie does **not** silently fall back to writing a sidecar. The UI shows a note that the tag is stored in Magpie's library only, but the tag is still persisted in the DB so filtering / searching keeps working.

Legacy sidecar files

Older Magpie versions and other tools (Lightroom, digiKam) may have left `Photo.xmp` files next to some images. Magpie's reader still picks these up on scan so users don't lose data. On the first successful save into the source file, `write_metadata_to_source` best-effort-deletes the leftover sidecar — after that the photo is the sole source of truth.

Sidecar path detection follows the Lightroom convention: `IMG_1234.jpg` → `IMG_1234.xmp` (strip original extension). The alternative `IMG_1234.jpg.xmp` (digiKam) form is not read.

Precedence when both exist (read path only)

If both the file's embedded XMP and a legacy sidecar exist, `read_all` merges embedded first and lets the sidecar overwrite, so the sidecar wins. This matches what Lightroom does and means an existing sidecar authored elsewhere still takes effect until Magpie saves and cleans it up.

Layer 2: the SQLite index

Location: `%APPDATA%\com.magpie.app\library.db`.

The database has six user tables plus `_migrations` and the FTS5 virtual table `images_fts`. Full schema in [Database schema](#).

Key properties:

- **Single writer** via a `Mutex<Connection>`. All writes are transactional; a failure rolls back cleanly.
- **FTS5 in contentless_delete=1 mode**. This is a SQLite 3.43+ feature that lets us `DELETE FROM images_fts` before re-inserting a row — necessary because we rebuild an FTS row on every metadata update. See migration `0002_fts_contentless_delete` in `src-tauri/src/db/migrations.rs`.
- `meta_read_at` / `meta_written_at` columns on `images` are the pivot for FS synchronisation: if the source file's (or any legacy sidecar's) mtime is newer than `meta_read_at`, Magpie re-reads the FS before serving a detail request.

Layer 3: thumbnail cache

Location: `%APPDATA%\com.magpie.app\thumbs\`.

Format: WebP with quality 80, two sizes per image (`<id>-small.webp`, `<id>-medium.webp`). Small is ~256 px on the long edge, medium ~512 px.

Regeneration:

- On scan, if a thumbnail is missing or the source file's `mtime` is newer than the thumbnail's, Magpie regenerates.
- On delete, `delete_thumbnails(cache_dir, image_id)` removes every size.
- The user can safely delete the whole `thumbs\` folder; Magpie regenerates on next request.

Volume assumptions

- Library folders can live on any local volume, including NTFS with long paths (`\\?\` - prefixed) and OneDrive-mounted trees.
- The database and cache always live on `%APPDATA%` — the OS default local storage. Users who want to move it to another volume should create a junction; Magpie does not have a “move library” UI in v1.

Backup story

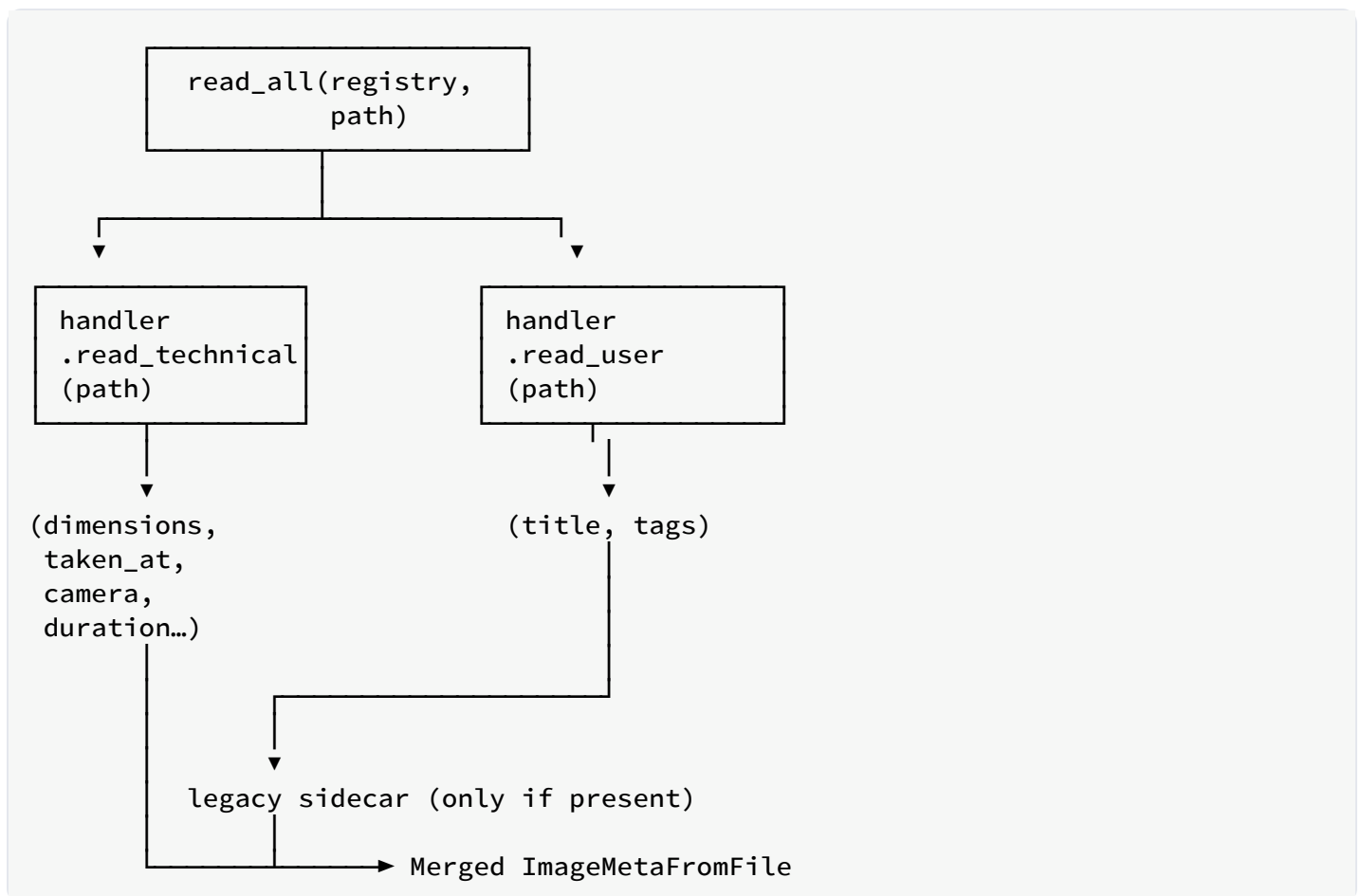
Everything that matters is on the filesystem outside Magpie's control: your original photos, with the metadata embedded inside them. Back those up and your library is safe. The SQLite database is regenerable and doesn't need to be in a backup — but including it means a restore doesn't require a rescan.

Metadata pipeline

The metadata pipeline is where Magpie most differs from a plain file browser. It has three sub-pipelines (read, patch, write) and a single invariant that ties them together:

Invariant. After a successful save via a writable [format handler](#), the database and the embedded metadata inside the source file reflect the same title and tags. Magpie does not produce sidecar files; any pre-existing legacy sidecar is removed by the write path after the embed succeeds. For files whose handler is read-only, the DB row and the library are the sole source of truth for tags.

The read pipeline



Ordering within user meta: the handler's own `read_user` is called first; then any **legacy** sidecar (from an older Magpie version or from Lightroom) is parsed and its non-empty fields overwrite

the handler's values. This preserves user data during the one-way migration to embed-only. The next successful save embeds the merged state and removes the sidecar.

The XMP parser (`quick_xml` state machine in `core/formats/xmp_packet.rs`) handles both the Adobe standard fields and Microsoft-Explorer variants:

- `dc:title`, `dc:description`, `dc:subject` (Alt/Bag containers)
- `xmp:Rating` (attribute or element — read for preservation, not surfaced in the UI)
- `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer's tag store)
- Attribute-only forms (some tools flatten Alt into an attribute)

The patch pipeline

Frontend produces a `MetadataPatch` — a struct where every field is `Option`-wrapped so the caller can express “don't touch” vs. “clear” vs. “set”:

```
pub struct MetadataPatch {
    pub title: Option<Option<String>>,
    pub tags: Option<Vec<String>>,          // whole list, replaces
    pub tags_add: Option<Vec<String>>,      // additive
    pub tags_remove: Option<Vec<String>>,
}
```

`apply_metadata_patch` (in `db/queries.rs`) runs the whole patch in a single transaction:

1. Update the `images` row title.
2. If `tags` is set: replace the row's tags.
3. If `tags_add` is set: insert missing.
4. If `tags_remove` is set: delete matching.
5. Rebuild the FTS5 row (DELETE + INSERT).
6. Commit.

If any step fails, the transaction rolls back. Nothing partially lands.

The write pipeline

After `apply_metadata_patch` returns success, the caller (`update_image_metadata` or `batch_update_metadata` via `apply_patch_and_persist`) does:

Fetch the **final** state via `get_image`
(reflects the just-applied patch)



```
spawn_blocking →  
write_metadata_to_source  
├ registry.for_ext(ext) → handler  
├ handler.write_user(path, meta)  
│   JPEG    → APP1 XMP  
│   PNG     → iTXt XMP chunk  
│   WebP    → RIFF XMP chunk  
│   GIF89a  → Application Extension  
│   else    → Err(unsupported)  
└ delete legacy .xmp if any
```



```
On Ok:  
set_meta_written_at + set_meta_read_at  
Emit app://image-updated  
  
On Err (writable handler failed):  
Propagate to caller (UI toast).  
  
On Err (handler is read-only):  
DB kept, UI shows "library only" note.
```

Three design choices worth calling out:

1. **The XMP packet is built from the DB's final state, not from the patch.** This way, "add tag X" in batch mode writes a packet containing every tag currently on the file, not just X.
2. **meta_read_at is bumped on write** so the FS-refresh check in `get_image` doesn't fire on files Magpie itself just wrote.
3. **File-write failure surfaces to the UI.** Unlike the old "sidecar is the fallback" design, there's no silent fallback: an unsupported format returns `Err` so the user sees a clear note. Any tag entered by the user is still recorded in the DB either way, so nothing they typed is lost.

FS refresh on read

Every `get_image` call does:

```
if refresh_needed_from_fs(&path, &cached) {  
    let fresh = read_all(&registry, &path);  
    resync_user_meta_from_fs(&db, id, &fresh);  
    set_meta_read_at_now(&db, id);  
}
```

`refresh_needed_from_fs` is a simple mtime comparison: if the source file (or any legacy `.xmp` alongside it) was modified after Magpie's last read, we re-read from disk. This is how a tag added in Windows Explorer becomes visible on next click of that file. The legacy-sidecar branch remains so that users migrating from Lightroom (or an older Magpie) still get their existing edits on first scan.

Windows Explorer tag interop specifics

Explorer writes tags into JPEGs using both `dc:subject` (standard) and `MicrosoftPhoto:LastKeywordXMP` (Microsoft-specific). Some older workflows only write the latter, so Magpie's reader accepts both and unions them, deduping case-insensitively.

On the write side, Magpie emits `dc:subject` in the XMP packet. Windows Explorer resolves its *Tags* column from either `dc:subject` or `MicrosoftPhoto:LastKeywordXMP`, so a Magpie-written JPEG shows up correctly in Explorer with only the standard Dublin Core block.

Preserving fields Magpie doesn't own

The XMP builder preserves any `dc:description`, `xmp:Rating`, GPS coordinates, and other tags a foreign tool wrote — Magpie's UI doesn't expose these, but the reader still parses them into `XmpUserMeta` so the writer can put them back into the rebuilt packet unchanged. See the `write_preserves_foreign_rating_and_description` integration test.

Observability

File-based logging

Every run of Magpie produces a rolling log file at:

```
%APPDATA%\com.magpie.app\logs\app.log
```

The logger is initialised in `src-tauri/src/lib.rs::init_logging`, using `tracing-subscriber` with a plain-text formatter and a `RUST_LOG`-compatible env filter (default: `info,desktop_lib=info`).

Format of a typical line:

```
2026-08-17T18:53:12.331332Z INFO desktop_lib: Magpie started app_data_dir="..."
2026-08-17T19:22:14.029142Z INFO desktop_lib::commands::images: get_image:
resynched user metadata from FS id=5
2026-08-18T08:41:03.912483Z INFO desktop_lib::commands::images:
batch_update_metadata count=3 patch=...
```

- ISO-8601 timestamp with microsecond precision.
- Log level: `TRACE` / `DEBUG` / `INFO` / `WARN` / `ERROR`.
- Target: usually the module path (`desktop_lib::commands::images`).
- Structured fields on the tail: `id=5` , `count=3` , `?patch` .

Structured fields

We use `tracing`'s field syntax pervasively:

```
tracing::info!(
    id,
    ?patch, // Debug-formatted
    op = "update_image_metadata", // static label
    "metadata embedded in source file"
);
```

The `?` prefix invokes `Debug`. Named fields keep the log grep-friendly: to find every batch save, `rg 'batch_update_metadata'` returns just the events, and the `id/count/count-succeeded`

triplet is on the same line.

Frontend crumbs into the same log

The renderer can push log lines into the Rust log via the `log_frontend` command (`src-tauri/src/commands/diag.rs`). The frontend helper `logFrontend(level, msg)` in `src/ipc.ts` is a best-effort fire-and-forget:

```
logFrontend('info', `applyTags dispatch: ids=${ids.length} add=[...]`)
```

Renderer-side events land in the log with a `target="frontend"` marker, which makes them easy to filter:

```
2026-08-18T08:41:03.900001Z INFO frontend: applyTags dispatch: ids=3 add=[vacation] remove=[]
```

This lets us reason about the full IPC round-trip from a single tail of `app.log`.

What’s logged where

Signal	Level	Emitted by
App started / logging initialised	INFO	<code>lib.rs::init_logging</code> , <code>lib.rs</code> main run
Migration applied	INFO	<code>db/migrations.rs::run</code>
Command entry (<code>update_image_metadata</code> , ...)	INFO	Each Tauri command that mutates state
<code>get_image</code> FS-refresh triggered	INFO	<code>commands/images.rs::get_image</code>
Embedded XMP write failed	WARN	<code>commands/images.rs::apply_patch_and_persist</code>
Unsupported format for embed	WARN	<code>core/metadata/write.rs::write_metadata_to_source</code>

Signal	Level	Emitted by
Legacy sidecar cleanup failed	WARN	<code>core/metadata/write.rs::write_metadata_to_sourc</code>
Scan errors (unreadable file, permission)	WARN	<code>core/scanner.rs</code>
DB migration or startup errors	ERROR	<code>db/mod.rs</code> , <code>lib.rs::run</code>
Frontend applyTags dispatch	INFO	<code>features/DetailsPanel.tsx::MultiDetails.applyTa</code>

Log rotation

The current implementation is a plain append-only file — no rotation. For a v1 release this is fine; the log grows a few kilobytes per session at INFO. A future PR should add `tracing-appender::rolling` with daily rotation or size-based (e.g. 10 MB × 5).

Turning up the volume

Set the `RUST_LOG` environment variable before launching:

```
$env:RUST_LOG = "debug,desktop_lib=trace"
& "$env:LOCALAPPDATA\Programs\Magpie\desktop.exe"
```

This will produce a torrent of scanner-level detail, useful when debugging a bad scan on a specific folder. ▶

No telemetry

There is no automatic upload of the log, no crash reporter, no analytics. `app.log` is on your disk and stays there unless you explicitly share it (e.g. attach it to a bug report).

Repository layout

```
Magpie/
├── src/
│   ├── App.tsx
│   ├── main.tsx
│   ├── index.css
│   ├── ipc.ts
│   ├── store.ts
│   ├── types.ts
│   └── features/
│       ├── TopBar.tsx
│       ├── search, sort
│       ├── Sidebar.tsx
│       ├── ImageGrid.tsx
│       ├── DetailsPanel.tsx
│       ├── (Title / Tags / Format / Info)
│       ├── TagInput.tsx
│       ├── Thumbnail.tsx
│       └── StatusBar.tsx
├── src-tauri/
│   ├── Cargo.toml
│   ├── tauri.conf.json
│   ├── build.rs
│   ├── capabilities/
│   │   └── default.json
│   ├── examples/
│   │   ├── dump_meta.rs
│   │   └── dump_tag_usage.rs
│   ├── tests/
│   │   └── metadata_fs.rs
│   └── src/
│       ├── main.rs
│       └── lib.rs
├── registration
│   ├── error.rs
│   ├── types.rs
│   ├── db/
│   │   ├── mod.rs
│   │   ├── migrations.rs
│   │   └── queries.rs
│   ├── core/
│   │   ├── mod.rs
│   │   ├── scanner.rs
│   │   ├── thumbnail.rs
│   │   ├── formats/
│   │   │   ├── mod.rs
│   │   │   ├── common.rs
│   │   │   └── xmp_packet.rs
│   └── truth)
│       ├── jpeg.rs
│       ├── # React frontend (TypeScript)
│       ├── # Root component; layout + event listeners
│       ├── # ReactDOM entry
│       ├── # Tailwind directives + global styles
│       ├── # Typed Tauri command wrappers
│       ├── # Zustand global UI state
│       ├── # Mirrors Rust types
│       ├── # Top-of-window: add folder, rescan,
│       ├── # Left: library / tag filters
│       ├── # Virtualised file grid
│       ├── # Right: SingleDetails / MultiDetails
│       ├── # Tag entry with autocompletion
│       ├── # <img> wrapper with async src
│       ├── # Bottom: scan progress, app version
│       ├── # Rust backend (Cargo package)
│       ├── # App config: identifier, security, window
│       ├── # Capability manifest (asset scopes, etc.)
│       ├── # Diagnostic: dump everything about a photo
│       ├── # Diagnostic: find photos tagged X
│       ├── # Integration tests for metadata pipeline
│       ├── # Windows subsystem entry – calls lib::run
│       ├── # Tauri builder, logging, handler
│       ├── # AppError, AppResult
│       ├── # Rust-side IPC types (mirror src/types.ts)
│       ├── # Db (Mutex<Connection>) + with_conn helper
│       ├── # SQL migrations (schema + FTS fix)
│       ├── # apply_metadata_patch, get_image, etc.
│       ├── # AppServices, FormatRegistry, dirs
│       ├── # Parallel folder scan
│       ├── # Thumb generation + caching
│       ├── # Per-format handlers (registry-based)
│       ├── # FormatHandler trait + FormatRegistry
│       ├── # Atomic write, EXIF/dims utilities
│       ├── # XMP parse + build (single source of
│       ├── # writable
```

```

├── png.rs                # writable
├── webp.rs               # writable
├── gif.rs                # writable (GIF89a only)
├── tiff.rs               # read-only
├── stubs.rs              # HEIC, PDF, video, RAW, ... (read-only)
├── metadata/
│   ├── mod.rs
│   ├── read.rs           # Delegates to registry + legacy sidecar
│   ├── write.rs          # Delegates to registry + sidecar cleanup
│   └── sidecar.rs        # Legacy `.xmp` path helper (read +
cleanup)
├── commands/            # Tauri command handlers
│   ├── mod.rs
│   ├── library.rs        # add/remove/list folders, rescan
│   ├── images.rs         # query, get, update, batch_update, delete
│   ├── tags.rs           # list / rename / delete tags
│   ├── collections.rs    # smart collections (skeleton in v1)
│   ├── thumbs.rs        # thumb + asset path resolvers
│   └── diag.rs           # log_frontend bridge
├── scripts/
│   ├── screenshot-app.ps1 # Verify UI screenshot
│   ├── screenshot-scrolled.ps1 # Screenshot with programmatic scroll
│   └── test-multiselect-tag.ps1 # E2E UI test skeleton
├── docs/                # This documentation
│   ├── user-manual/      # mdBook: user-facing manual
│   ├── developer/        # mdBook: this guide
│   ├── shared/           # Shared CSS / JS for both books
│   ├── index.html        # Landing page linking both docs
│   └── build.ps1         # HTML + PDF build script
├── index.html            # Vite entry
├── vite.config.ts
├── tailwind.config.js
├── postcss.config.js
├── tsconfig.json
├── package.json
├── README.md
└── .gitignore

```

What lives where

- **Anything user-visible** (button text, layout, animations) is under `src/`.
- **Anything touching disk** (files, DB) is under `src-tauri/src/`. If you find frontend code doing FS work, that's a bug.
- **Anything that runs at build time** (Tauri config, capabilities, Cargo settings) is under `src-tauri/` outside `src/`.

- **Documentation source** is under `docs/` . Generated HTML lands in `docs/user-manual/book/` and `docs/developer/book/` ; PDFs land in `docs/` .

Backend modules

lib.rs — app entry

Responsibilities:

- Initialise file-based logging (`init_logging`).
- Build `AppServices { db, cache_dir, ... }` under `Arc`.
- Register every Tauri plugin (`dialog`, `opener`) and command handler in `invoke_handler!`.
- Manage the app run loop.

Notable pitfalls:

- `init_logging` **must not** call `tracing_subscriber::fmt().init()` if a Tauri plugin (like the removed `tauri-plugin-log`) already set a global subscriber. The prior implementation crashed with `PluginInitialization("log", "attempted to set a logger after...")`; the current version owns the logger outright.

types.rs — IPC types

- `ImageSummary`, `ImageDetails` — full and abridged views of an image row.
- `MetadataPatch` — the “what to change” payload for `update_image_metadata` and `batch_update_metadata`.
- `double_option` module — custom Serde deserializer for `Option<Option<T>>`. Distinguishes “field missing” from “field explicitly null” on the wire.

Every struct uses `#[serde(rename_all = "camelCase")]` so JSON keys are camelCase but Rust fields stay snake_case.

error.rs

`AppError` enum (via `thiserror`):

- `Io(std::io::Error)` — filesystem errors.

- `Db(rusqlite::Error)` — DB errors.
- `MetadataRead(String)`, `MetadataWrite(String)` — user-facing metadata failures.
- `Scan(String)` — scanner-specific issues.
- `Internal(String)` — anything else.

All commands return `AppResult<T>` (= `Result<T, AppError>`). The `Display` impl for `AppError` is safe to surface to the user (no absolute paths in strings without context).

db/mod.rs

Thin wrapper around `rusqlite::Connection`:

```
pub struct Db {
    conn: Arc<Mutex<Connection>>,
}

impl Db {
    pub fn open(path: &Path) -> AppResult<Self> { ... }

    pub fn with_conn<F, T>(&self, f: F) -> AppResult<T>
    where
        F: FnOnce(&Connection) -> AppResult<T>,
    { ... }
}
```

The single-mutex approach keeps SQLite calls simple and avoids connection pooling. WAL mode is enabled on open, so concurrent readers don't stall writers.

db/migrations.rs

Two migrations in the ordered `MIGRATIONS` slice:

1. `0001_init` — full schema (see [Database schema](#)).
2. `0002_fts_contentless_delete` — recreate `images_fts` with `contentless_delete=1` and blank out `meta_read_at` so the FS sync path re-runs for every image.

Migrations run at startup inside `run(conn)`. Each is applied atomically and recorded in the `_migrations` table.

db/queries.rs

The functional heart of the DB layer. Highlights:

Function	Purpose
<code>apply_metadata_patch(db, id, p)</code>	Apply a <code>MetadataPatch</code> to an image in a single txn.
<code>get_image(db, id)</code>	Fetch full <code>ImageDetails</code> including tags.
<code>query_images(db, filter, sort, page)</code>	Paginated grid query with sidebar filters + FTS.
<code>list_tags(db, prefix)</code>	Tags with counts, optionally prefix-filtered.
<code>rename_tag(db, old, new)</code>	Global rename in one txn (including FTS rebuild).
<code>delete_tag(db, name)</code>	Global delete of a tag.
<code>get_image_paths(db, ids)</code>	Bulk fetch of paths for delete.
<code>delete_image_rows(db, ids)</code>	Bulk row deletion after files are gone.
<code>resync_user_meta_from_fs(db, id, meta)</code>	Sync FS-read metadata into DB, not touching
	scan-derived fields like size/mtime.
<code>set_meta_read_at_now(db, id)</code>	Timestamp helper.
<code>set_meta_written_at(db, id, t)</code>	Timestamp helper.

Every mutation goes through a `Transaction` obtained via `conn.transaction()?`, and every one calls `rebuild_fts_row_tx` at the end to keep the search index in sync.

core/scanner.rs

Pipeline for `add_library_folder` and `rescan_*`:

1. **Walk.** `jwalk::WalkDir` traverses the folder in parallel.
2. **Filter.** Extensions checked against `IMAGE_EXTS` (see `core/mod.rs`).
3. **Diff.** For each file, look up by path in `images`; skip if `mtime_ms` matches.

4. **Extract.** In parallel via `rayon`: EXIF read, XMP read, content hash (`xxh3`).
5. **Upsert.** DB writes are serialised via the single connection mutex; scanner buffers batches to amortise txn overhead.
6. **Thumbnails.** Enqueue small + medium.
7. **Progress.** Emit `app://scan { done, total, current }`.

Scanning is bounded by disk read speed on cold cache and by CPU on warm cache.

core/thumbnail.rs

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, image_id: i64) ->
Result<>>;
pub fn thumb_path(cache_dir: &Path, image_id: i64, size: ThumbSize) -> PathBuf;
pub fn delete_thumbnails(cache_dir: &Path, image_id: i64);
```

Decode via `image::open`, resize via `fast_image_resize` (SIMD Lanczos3), encode via `webp::Encoder` at quality 80.

core/metadata/read.rs

`read_all(path) -> ImageMeta` runs the read pipeline:

1. `exif::Reader::read_from_container` for taken time + camera fields.
2. `xmp::extract_embedded_xmp(path)` for embedded XMP (JPEG APP1 or PNG iTXt).
3. Read any legacy `<image>.xmp` sidecar for backward compatibility.
4. Merge sidecar-over-embedded via `apply_user_meta` (sidecar wins for user metadata so a Lightroom-authored `.xmp` still takes effect on first scan).

Non-fatal errors are collected into a per-file warning log; the scanner continues on the next file.

core/formats/

The format handler framework. Each supported extension is owned by one `FormatHandler` implementation:

- `mod.rs` — declares the `FormatHandler` trait, `TechnicalMeta`, `UserMeta`, `FormatKind`, and the `FormatRegistry` that lives on `AppServices`.

- `xmp_packet.rs` — a hand-written streaming XMP reader/writer. Extracts and rebuilds packets containing title, description, rating, subjects, and Microsoft-Photo keywords. The description and rating fields are preserved on read-modify-write even though Magpie's UI no longer surfaces them.
- `common.rs` — shared `atomic_write_bytes`, EXIF → technical metadata, dimensions, and `write_not_supported_error` (for read-only stubs).
- `jpeg.rs`, `png.rs`, `webp.rs`, `gif.rs` — writable handlers. Each parses the format's container, drops any old XMP block, and splices in a freshly-built packet.
- `tiff.rs`, `stubs.rs` — read-only handlers for TIFF, HEIC, PDF, video, camera RAW, BMP, EXR, HDR, SVG, and more.

Windows Explorer's *Tags* column resolves from either `dc:subject` or `MicrosoftPhoto:LastKeywordXMP`, so Magpie only emits the standard Dublin Core form.

core/metadata/write.rs

`write_metadata_to_source` is the single entry point for saves and is now a thin façade over the registry:

1. **Handler lookup.** `registry.for_ext(ext).ok_or(UnsupportedFormat)?`.
2. **Handler write.** `handler.write_user(path, &meta)?`. The handler:
 - reads current on-disk state,
 - merges the incoming edits with `xmp_packet::merge_user_edits` (preserving foreign fields),
 - rebuilds the packet,
 - splices it into the container,
 - writes atomically via `common::atomic_write_bytes`.
3. **Best-effort delete** of any leftover `<file>.xmp` from a previous Magpie version or from Lightroom.

Failure semantics:

- Unsupported format = `Err(_)` before any disk touch (read-only handler's `write_user` returns immediately).
- Embed failure (read-only file, disk full, corrupt file) = `Err(_)`.
- Sidecar cleanup failure = logged as WARN, save still returns `ok(())` because the source file already carries the metadata.

commands/*.rs

Each file exposes 3–5 `#[tauri::command]` async functions. See [Tauri command reference](#) for the full list.

Common patterns:

- `services: State<'_, Arc<AppServices>>` for shared state.
- `app_handle: AppHandle` for emitting events.
- Return `AppResult<T>`; Serde does the JSON legwork.
- Tracing spans on entry so `app.log` shows every IPC hit.

Frontend modules

The frontend is intentionally small — the goal is to be a translation layer between UI events and Tauri commands, not to hold domain logic.

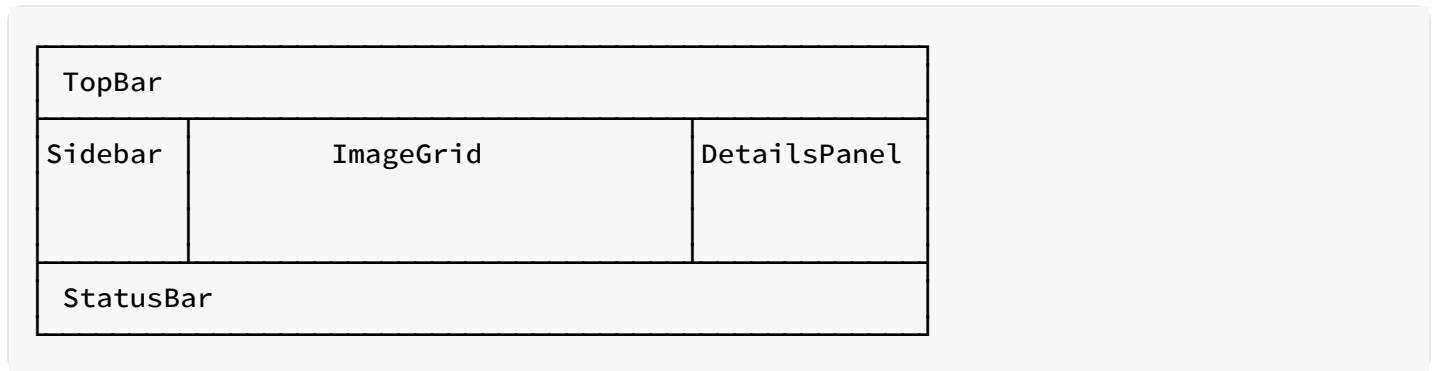
src/main.tsx

Boots React. Wraps `<App />` in a `QueryClientProvider` with a single `QueryClient` configured for:

- `staleTime: 0` — refetch on refocus.
- `retry: 1` — one retry on network / IPC errors.
- Query cache keys convention (below).

src/App.tsx

Owns the top-level layout:



Also hosts the global keyboard listener (Delete key), and mounts event listeners for `app://image-updated` and `app://images-deleted` — both of which invalidate the relevant query keys.

src/store.ts (Zustand)

Global UI-only state:

```
interface Store {
  view: View // 'all' | { folderId } | { tag }
  search: string
  sort: ImageSort
  extraFilter: ImageFilter // sidebar filters composed
  selection: { ids: Set<number>; anchor: number | null }
  detailsOpen: boolean
  // setters
  setView, setSearch, setSort, setSelection, clearSelection, ...
}
```

No persistence in v1 — reloading the app resets to `view: 'all'`, empty search, no selection.

`filterFromView(view, extra, search)` composes the sidebar view, the extra filter, and the search string into a single `ImageFilter` sent to `query_images`.

src/ipc.ts

Thin, typed wrappers around `invoke`:

```
export const updateImageMetadata = (id: number, patch: MetadataPatch) =>
  invoke<ImageDetails>('update_image_metadata', { id, patch })

export const batchUpdateMetadata = (ids: number[], patch: MetadataPatch) =>
  invoke<number[]>('batch_update_metadata', { ids, patch })

// ...
```

Plus event-listener helpers:

```
export const onImageUpdated = (h: (id: number) => void) =>
  listen<number>('app://image-updated', e => h(e.payload))
```

And the diagnostic helper `logFrontend(level, msg)` for pushing crumbs into the backend log.

src/types.ts

Every IPC struct duplicated from Rust, kept in sync manually. See [IPC boundary](#) for the rationale (no derived types).

src/features/

TopBar.tsx

- Add folder button (opens Tauri dialog, calls `add_library_folder`, invalidates `['folders']` and `['images']`).
- Rescan button (calls `rescan_all`).
- Live search input (debounced 200 ms → `useStore.setSearch`).
- Sort dropdown + direction toggle.
- Toggle-details icon.

Sidebar.tsx

- `['folders']`, `['tags']` queries fed to three collapsible sections.
- Click handlers set `useStore.view` and clear other filters.
- Right-click context menu for `Remove folder`, `Rescan folder`, `Rename tag`, `Delete tag`.

ImageGrid.tsx

- `useQuery(['images', filter, sort, page])` returns paginated results.
- `useVirtualizer` from TanStack Virtual computes visible rows.
- Each tile is a `Thumbnail` component; `Ctrl / Shift / plain` click logic mutates `useStore.selection`.

DetailsPanel.tsx

- Reads `useStore.selection` and dispatches:
 - 0 selected → placeholder.
 - 1 selected → `<SingleDetails id=... />`.
- 1 selected → `<MultiDetails ids=... />`.

`SingleDetails` renders four fixed sections:

1. **Title** — editable input, auto-saves via `updateImageMetadata`.

2. **Tags** — `TagInput`, auto-saves via `updateImageMetadata`.
3. **Format metadata** — read-only: handler name + `canWriteTags` note. Room to grow into per-format editable fields (GPS, description, ...) as those handlers add them.
4. **File info** — `<dl>` of the `technical` list returned by the backend.

Local edit state is seeded from the query result only when the `id` changes (guarded by `lastLoadedId.current`), avoiding the “stomp typing on refetch” bug.

`MultiDetails` shows two `TagInput`s (Add / Remove) and an Apply button. It uses refs (`tagsAddRef`, `tagsRemoveRef`) mirroring state so the mutation dispatch always sees the latest values, even if a blur-triggered `setTagsAdd` and a click on `Apply` land in the same React batch.

TagInput.tsx

- Controlled by parent's `tags: string[]` + `onChange(next: string[])`.
- Commits the current draft on Enter, Space, Comma, or blur.
- Autocompletion pulled from `list_tags(prefix)` via TanStack Query.
- Backspace on empty input pops the last tag.

Thumbnail.tsx

- Resolves `getThumbPath(id, 'small')` on mount, sets `<img src>` to the returned asset URL.
- Placeholder while the thumbnail is being generated (rare, only on first scan).

StatusBar.tsx

- Listens to `app://scan` events, shows a live progress bar and message.
- Static labels for app version, DB size, thumbnail cache size.

Query key conventions

Query key	Fetches by	Invalidated by
<code>['folders']</code>	<code>listLibraryFolders</code>	add/remove/rescan folder, delete images

Query key	Fetches by	Invalidated by
<code>['tags']</code>	<code>listTags</code>	any metadata update, tag rename/delete
<code>['tag', prefix]</code>	<code>listTags(prefix)</code>	any metadata update
<code>['images', filter, sort, page]</code>	<code>queryImages</code>	any image mutation
<code>['image', id]</code>	<code>getImage</code>	<code>update_image_metadata</code> (via <code>setQueryData</code>),
		<code>batch_update_metadata</code> (via <code>invalidateQueries</code>),
		<code>app://image-updated</code> event
<code>['smartCollections']</code>	<code>listSmartCollections</code>	create/delete collection

Styling

- Tailwind for utility classes.
- Global styles in `src/index.css`: dark background, rounded buttons, focus rings.
- No CSS-in-JS. Component-scoped styles are className concatenations via `clsx` when needed.

Database schema

The database is a single SQLite file at `%APPDATA%\com.magpie.app\library.db`. Schema definition lives in `src-tauri/src/db/migrations.rs`.

Tables

`library_folders`

Every folder the user has added to the library.

Column	Type	Notes
id	INTEGER	PK, autoincrement.
path	TEXT	Absolute, canonical, UNIQUE.
added_at	INTEGER	Unix ms.
last_scan_at	INTEGER	Unix ms; NULL before first scan finishes.

`images`

One row per file (of any format the registry recognises) found by the scanner. The table is still called `images` for historical reasons; a future migration may rename it to `files`.

Column	Type	Notes
id	INTEGER	PK, autoincrement.
folder_id	INTEGER	FK → <code>library_folders(id)</code> ON DELETE CASCADE.
path	TEXT	Absolute path, UNIQUE.
filename	TEXT	Basename (<code>IMG_1234.jpg</code>).
ext	TEXT	Lowercase extension, no dot (<code>jpg</code> , <code>heic</code> , ...).
size_bytes	INTEGER	From <code>fs::Metadata::len()</code> .

Column	Type	Notes
mtime_ms	INTEGER	Modification time in Unix ms.
width	INTEGER	Nullable; from handler's technical read.
height	INTEGER	Nullable.
content_hash	TEXT	Nullable; XXH3 128-bit hex digest.
taken_at	INTEGER	Nullable; Unix ms from EXIF <code>DateTimeOriginal</code> .
camera_make	TEXT	Nullable.
camera_model	TEXT	Nullable.
title	TEXT	Nullable; user-editable via UI.
meta_written_at	INTEGER	Nullable; last time Magpie wrote XMP for this file.
meta_read_at	INTEGER	Nullable; last time Magpie read XMP from disk into DB.
missing	INTEGER	0/1 flag for "file expected but not found" (unused v1).

Migration `0003_drop_rating_comment` removed the `rating` and `comment` columns (v1 shipped without them in the UI).

Indexes:

- `idx_images_folder` (`folder_id`)
- `idx_images_taken_at` (`taken_at`)
- `idx_images_filename` (`filename`)

tags

The tag vocabulary.

Column	Type	Notes
id	INTEGER	PK, autoincrement.
name	TEXT	UNIQUE COLLATE NOCASE.

image_tags

Many-to-many join between `images` and `tags`.

Column	Type	Notes
image_id	INTEGER	FK → <code>images(id)</code> ON DELETE CASCADE.
tag_id	INTEGER	FK → <code>tags(id)</code> ON DELETE CASCADE.
PK		(image_id, tag_id).

Indexes:

- `idx_image_tags_tag` (tag_id) — accelerates “photos with tag X”.

images_fts (virtual, FTS5)

Full-text search index over three columns.

```
CREATE VIRTUAL TABLE images_fts USING fts5(  
  title, filename, tags,  
  content='',  
  contentless_delete=1,  
  tokenize='unicode61 remove_diacritics 2'  
);
```

Key attributes:

- **Contentless** (`content=''`): the FTS table doesn't store the original text; queries return `rowid` (= `images.id`) and we join.
- **contentless_delete=1** (SQLite 3.43+): lets us `DELETE FROM images_fts` before re-inserting a row. Without it, `rebuild_fts_row_tx` fails and rolls back every metadata write. See migration `0002_fts_contentless_delete`.
- **Diacritics removed at level 2**: `naive` and `naïve` are the same token.

smart_collections

Skeleton in v1 — data model exists, UI to come.

Column	Type	Notes
id	INTEGER	PK, autoincrement.

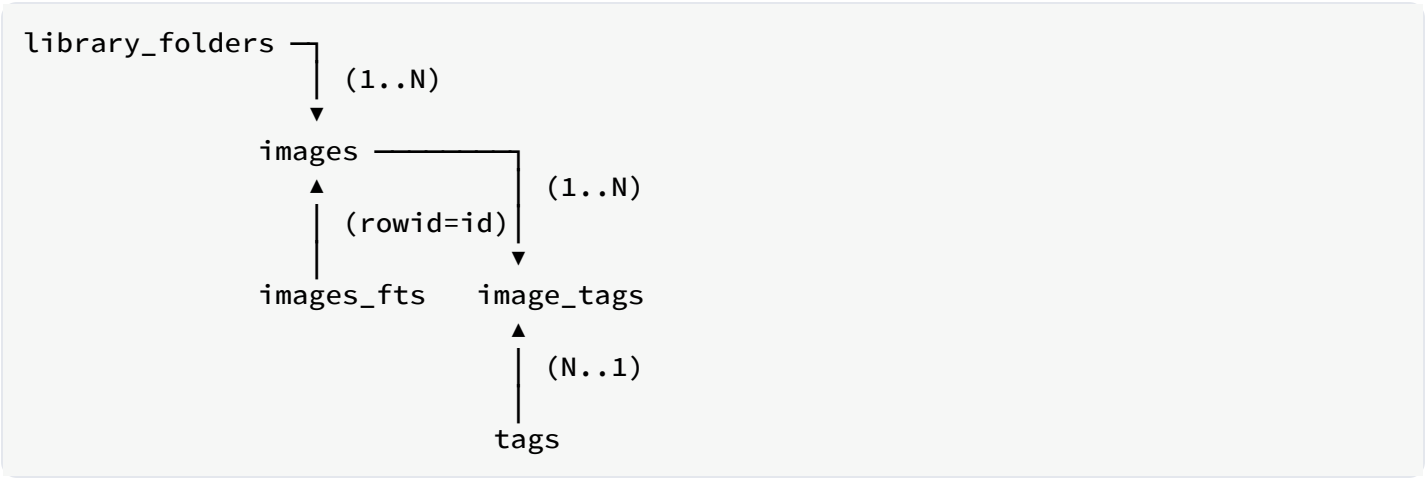
Column	Type	Notes
name	TEXT	
filter	TEXT	JSON-serialised <code>ImageFilter</code> .
sort_order	INTEGER	

_migrations

Housekeeping.

Column	Type	Notes
name	TEXT	PK; migration name (e.g. <code>0001_init</code>).
applied_at	INTEGER	Unix ms when the migration ran.

Relationships



Migrations

Name	Effect
<code>0001_init</code>	Full schema above (with <code>rating</code> / <code>comment</code> , without <code>contentless_delete</code>).

Name	Effect
0002_fts_contentless_delete	Drop + recreate <code>images_fts</code> with <code>contentless_delete=1</code> ;
	repopulate from live data; blank <code>meta_read_at</code> to force
	FS re-read.
0003_drop_rating_comment	Drop <code>rating</code> and <code>comment</code> columns from <code>images</code> ; drop
	<code>comment</code> column from <code>images_fts</code> ; strip rating/comment
	filters from any stored smart collections; blank
	<code>meta_read_at</code> to force FS re-read on next open.

Each migration is applied atomically inside a transaction and recorded in `_migrations` on success.

Tauri command reference

Every command below is registered in `src-tauri/src/lib.rs` inside `tauri::generate_handler! [...]`. All are `async fn`, return `AppResult<T>`, and use camelCase JSON on the wire.

Library management

add_library_folder

```
async fn add_library_folder(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
    path: String,  
) -> AppResult<LibraryFolder>;
```

Add a folder to the library and start a background scan. Emits `app://scan` progress events. Returns the created row.

remove_library_folder

```
async fn remove_library_folder(  
    services: State<'_, Arc<AppServices>>,  
    id: i64,  
) -> AppResult<()>;
```

Removes the folder row (CASCADE removes every image and image_tags row). Files on disk are untouched.

list_library_folders

```
async fn list_library_folders(  
    services: State<'_, Arc<AppServices>>,  
) -> AppResult<Vec<LibraryFolder>>;
```

rescan_folder

```
async fn rescan_folder(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
    id: i64,  
) -> AppResult<ScanResult>;
```

Re-walks the specified folder. Incremental: only files with a newer `mtime` than what's in the DB get re-processed.

rescan_all

```
async fn rescan_all(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
) -> AppResult<Vec<ScanResult>>;
```

Images

query_images

```
async fn query_images(  
    services: State<'_, Arc<AppServices>>,  
    filter: Option<ImageFilter>,  
    sort: Option<ImageSort>,  
    page: Option<Pagination>,  
) -> AppResult<Page<ImageSummary>>;
```

Paginated grid query. `filter` composes:

- `folder_id: Option<i64>`
- `tag: Option<String>`
- `search: Option<String>` — passed through FTS5.

Returns `Page { items, total, page, page_size }`.

get_image

```
async fn get_image(  
    services: State<'_, Arc<AppServices>>,  
    id: i64,  
) -> AppResult<ImageDetails>;
```

Returns full details for one image. **Side effect:** if the source file's mtime (or any legacy `.xmp` sidecar's mtime) is newer than `meta_read_at`, re-reads metadata from disk and updates the DB *before* returning.

update_image_metadata

```
async fn update_image_metadata(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
    id: i64,  
    patch: MetadataPatch,  
) -> AppResult<ImageDetails>;
```

Apply a metadata patch, embed the merged XMP into the source file, delete any legacy `.xmp` sidecar, and return the new state. Emits `app://image-updated`. Returns `Err` on unsupported formats (anything other than JPEG or PNG) or file-write failures.

batch_update_metadata

```
async fn batch_update_metadata(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
    ids: Vec<i64>,  
    patch: MetadataPatch,  
) -> AppResult<Vec<i64>>;
```

Same as `update_image_metadata` but for many photos. Returns the list of IDs that succeeded; failures are logged and skipped.

delete_images

```
async fn delete_images(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
    ids: Vec<i64>,  
    permanent: Option<bool>,  
) -> AppResult<DeleteResult>;
```

Move to Recycle Bin (default) or permanently delete. Also removes any legacy `.xmp` sidecar + thumbnails + DB row. Returns per-file success/failure.

Tags

list_tags

```
async fn list_tags(  
    services: State<'_, Arc<AppServices>>,  
    prefix: Option<String>,  
) -> AppResult<Vec<TagStats>>;
```

Every tag with a photo count. Optional prefix filter for autocomplete.

rename_tag

```
async fn rename_tag(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
    old_name: String,  
    new_name: String,  
) -> AppResult<()>;
```

Global rename across every photo. Rewrites every affected embedded XMP packet in the source files.

delete_tag

```
async fn delete_tag(  
    services: State<'_, Arc<AppServices>>,  
    app_handle: AppHandle,  
    name: String,  
) -> AppResult<()>;
```

Remove a tag from every photo. Files are updated the same way as rename.

Smart collections (skeleton)

list_smart_collections

```
async fn list_smart_collections(  
    services: State<'_, Arc<AppServices>>,  
) -> AppResult<Vec<SmartCollection>>;
```

create_smart_collection

```
async fn create_smart_collection(  
    services: State<'_, Arc<AppServices>>,  
    name: String,  
    filter: ImageFilter,  
) -> AppResult<SmartCollection>;
```

delete_smart_collection

```
async fn delete_smart_collection(  
    services: State<'_, Arc<AppServices>>,  
    id: i64,  
) -> AppResult<()>;
```

Thumbnails and image paths

get_thumb_path

```
async fn get_thumb_path(  
    services: State<'_, Arc<AppServices>>,  
    id: i64,  
    size: ThumbSize,  
) -> AppResult<String>;
```

Returns the absolute path of a cached thumbnail; generates it on demand if missing.

get_image_path

```
async fn get_image_path(  
    services: State<'_, Arc<AppServices>>,  
    id: i64,  
) -> AppResult<String>;
```

Returns the absolute path of the source image (used by the details panel to render a large preview via `convertFileSrc`).

Diagnostics

log_frontend

```
fn log_frontend(level: String, msg: String) -> AppResult<()>;
```

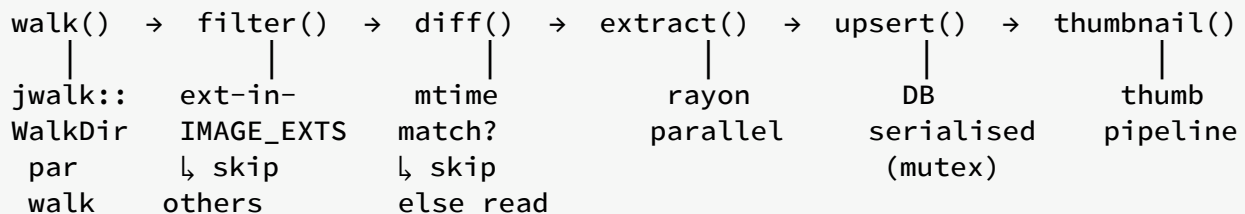
Push a log line into `app.log` from the renderer. `level` is one of `debug|info|warn|error`. Emitted with `target="frontend"` for easy filtering.

Scanner algorithm

Objective

Given a library folder path, populate the `images` table with a row per supported image file, extract metadata, generate thumbnails, and keep the DB in sync with the filesystem on subsequent runs — all while staying responsive.

Pipeline



Progress events (`app://scan { folder_id, done, total, current }`) are emitted every N files (default 20).

Stage 1 — walk

`jwalk::WalkDir::new(root).parallelism(Parallelism::RayonNewPool(n))` walks the folder tree in parallel across all cores, streaming `DirEntry` values. Symlinks are followed (with cycle detection); hidden files are respected per OS convention.

Stage 2 — filter

An entry is kept iff:

- It's a regular file (not a directory, socket, or device).
- Its lowercase extension is in `core::IMAGE_EXTS: jpg jpeg jpe jfif jif png gif bmp tif tiff webp heic heif cr2 cr3 nef arw dng raf orf pef x3f`.

- Its filename doesn't start with `.` (dotfiles).

Discarded entries increment a counter used for the progress denominator but otherwise do no work.

Stage 3 — diff

For each kept entry, Magpie looks up the path in `images`:

```
SELECT id, mtime_ms FROM images WHERE path = ?1
```

- **New file** (no row): full extract required.
- **Existing, mtime unchanged**: skip.
- **Existing, mtime changed**: partial re-extract (metadata + hash), leaving other columns intact.

The lookup is $O(\log n)$ thanks to the UNIQUE index on `images.path`.

Stage 4 — extract

For each file that needs work, the following runs on a rayon worker:

1. **Stat** — `fs::metadata(path)` for `size_bytes` and `mtime_ms`.
2. **Content hash** — stream the file through `xxh3_128`. On an SSD this is disk-bound at ~2 GB/s; on HDD it's the bottleneck.
3. **Handler lookup** — `registry.for_ext(ext)` picks the `FormatHandler` for the file's extension.
4. **Technical read** — the handler's `read_technical` produces ordered key/value pairs; those we recognise fold into `taken_at`, `camera_make`, `camera_model`, `width`, `height`.
5. **User meta** — `metadata::read::read_all` combines the handler's `read_user` with any legacy sidecar for `title` and `tags`.

Any per-file failure is logged (`WARN`) and doesn't abort the folder scan. The file gets a row with whatever metadata succeeded; missing fields are left NULL.

Stage 5 — upsert

The DB is the single writer. To amortise transaction overhead, extracted rows are pushed to a bounded channel; a single “writer task” drains the channel and commits batches of up to 128 rows in one transaction:

```
let tx = conn.transaction()?;
for row in batch { upsert_image(&tx, &row)?; }
tx.commit()?;
```

The `upsert_image` SQL is:

```
INSERT INTO images (path, folder_id, filename, ext, size_bytes, mtime_ms,
                    width, height, content_hash, taken_at,
                    camera_make, camera_model, title,
                    meta_read_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
ON CONFLICT(path) DO UPDATE SET
    size_bytes      = excluded.size_bytes,
    mtime_ms        = excluded.mtime_ms,
    width           = excluded.width,
    height           = excluded.height,
    content_hash     = excluded.content_hash,
    taken_at        = excluded.taken_at,
    camera_make      = excluded.camera_make,
    camera_model     = excluded.camera_model,
    title            = excluded.title,
    meta_read_at     = excluded.meta_read_at;
```

After the row lands, tags are reconciled: everything in `image_tags` for this image is deleted, then re-inserted from the fresh tag list. Both happen in the same transaction as the image upsert.

The FTS5 row is rebuilt via `rebuild_fts_row_tx` at the very end.

Stage 6 — thumbnails

Once a row is in the DB, the scanner enqueues a thumbnail task for that image id. Thumbnails run on the same rayon pool but with lower priority so they don't starve extraction.

```
ensure_thumbnails(cache_dir, src_path, image_id):
```

1. For each size (small, medium):
 - Path = `cache_dir / "<id>-<size>.webp"`.

- Skip if the thumbnail's mtime $\geq$ the source's mtime.
- 2. Decode the source with `image::open`.
- 3. Resize with `fast_image_resize::Resizer` (Lanczos3, SIMD).
- 4. Encode with `webp::Encoder::new(&image).encode(80)`.
- 5. Write bytes atomically (temp + rename).

Failures at any stage are logged and don't affect the DB row.

Handling deletions

If a file that was in the DB is no longer found on disk during a scan, the scanner does **not** delete the row automatically — the `missing` flag in the `images` table exists to mark it (v1: not exposed in UI). A future contribution can add an explicit “Clean up missing photos” UI action.

Incremental performance

The diff step is what makes rescans fast:

- 50 000 photos, no changes: `~2 seconds` (mostly the DB lookup and a stat call per file).
- 50 000 photos, 100 new: `~5 seconds` (mostly hashing + XMP read for the 100).
- 50 000 photos, cold cache (thumbnails missing): `~2 minutes` (bounded by encoding).

Thumbnail pipeline

Sizes

Two cached sizes per image, both stored as WebP quality 80:

Enum name	Long-edge px	Used by
Small	~256	ImageGrid tiles (main use case).
Medium	~512	(Reserved for a future zoom-in mode.)
Large	~1024	(Currently only generated on request.)

Aspect ratio is preserved — the short edge is proportional. A 6000×4000 source becomes a 256×170 small and 512×341 medium.

Location

Files live in `%APPDATA%\com.magpie.app\thumbs\`:

```
<id>-small.webp
<id>-medium.webp
```

Using the image id (rather than a hash of the source path) keeps the cache invariant across file renames: as long as Magpie still knows about the image, the thumbnail is reusable.

Generation

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, image_id: i64) ->
Result<()> {
    let src_mtime = fs::metadata(src)?.modified()?;
    for size in &[ThumbSize::Small, ThumbSize::Medium] {
        let out = thumb_path(cache_dir, image_id, *size);
        if is_fresh(&out, src_mtime) { continue; }
        let img = image::open(src)?;
        let target = pick_target_size(&img, *size);
        let thumb = resize_rgba(&img.to_rgba8(), target);
        save_webp(&thumb, &out)?;
    }
    Ok(())
}
```

Details:

- `image::open` handles JPEG, PNG, GIF, TIFF, WebP, HEIC (with the `image` crate's HEIC feature enabled). For unsupported RAW formats, `open` returns an error and the thumbnail is skipped (grid renders a placeholder).
- `resize_rgba` uses `fast_image_resize::Resizer` with the `Lanczos3` filter. On x86_64 this dispatches to SSE4/AVX2 SIMD automatically.
- `save_webp` writes to `<out>.tmp` then renames — atomic, so a crash mid-encode never leaves a corrupt WebP.

Freshness

A thumbnail is considered fresh iff its file mtime is $\geq$ the source file's mtime. Editing metadata does not change the source mtime enough to invalidate a thumbnail (unless Magpie's embed-XMP write touched the JPEG — in which case the thumbnail regenerates on next scan, which is correct because the JPEG has actually changed).

Deletion

`delete_thumbnails(cache_dir, image_id)` is called from `delete_images`. It removes every size:

```
for size in [Small, Medium, Large] {  
    let _ = fs::remove_file/thumb_path(cache_dir, image_id, size));  
}
```

Errors are ignored — a missing thumb is the desired state.

Cache sizing

The `Small` WebP averages 4–8 KB per photo; `Medium` averages 16–32 KB. Total cache size on a 50 000-photo library is roughly 1–2 GB.

There's no automatic eviction in v1. Users who want to trim the cache can delete `%APPDATA%\com.magpie.app\thumbs\` — Magpie regenerates on demand.

Async model

`ensure_thumbnails` is CPU-bound (decode + resize + encode). It's called from Tauri commands via `tauri::async_runtime::spawn_blocking` so it doesn't stall a Tokio worker. The scanner enqueues thumb tasks on the rayon pool to overlap I/O with compute.

Metadata read path

Entry point

```
pub fn read_all(registry: &FormatRegistry, path: &Path) ->
  AppResult<ImageMetaFromFile>;
```

Called from:

- The scanner (initial scan and rescans), to populate the DB.
- `get_image` (from a Tauri command), when a FS-refresh check detects that the source file (or any legacy sidecar) has changed since the DB last saw it.

`ImageMetaFromFile` is the union of everything we can pull from disk that the DB stores:

```
pub struct ImageMetaFromFile {
  pub title:      Option<String>,
  pub tags:       Vec<String>,      // deduped, in insertion order
  pub taken_at:   Option<i64>,      // ms since Unix epoch
  pub camera_make: Option<String>,
  pub camera_model: Option<String>,
  pub width:      Option<u32>,
  pub height:     Option<u32>,
}
```

Note: `rating` and `comment` were dropped in migration 0003; they are no longer surfaced in the DB or in `ImageDetails`. The XMP builder still preserves any `xmp:Rating` / `dc:description` a foreign tool wrote — see [Metadata write path](#).

Stages

1. Ask the handler

```
let handler = registry.for_ext(ext).ok_or(AppError::UnsupportedFormat)?;
let user     = handler.read_user(path)?;
let technical = handler.read_technical(path);
```

`read_user` returns the editable surface (title + tags). `read_technical` returns the ordered `TechnicalMeta` list used by the UI and by the scanner for dimensions / EXIF-derived fields (camera, taken_at). See [File formats](#).

Each handler decides how to fetch this. All writable handlers use the shared `xmp_packet::parse_xmp` helper; read-only stubs typically only implement `read_technical` and return an empty `UserMeta` from `read_user`.

2. Legacy sidecar XMP

```
let sidecar = read_sidecar(&sidecar_path_for(path)).ok();
```

Sidecar path is `<file basename>.xmp`. Sidecars are a **legacy compatibility** path: Magpie no longer produces them, but older Magpie installs and other tools (Lightroom, digiKam) did, and users would lose data if we ignored them. If a sidecar file exists it's parsed with the same XMP parser used for embedded packets.

3. Merge

`apply_user_meta` is called first with the handler's `UserMeta` (if any), then with the sidecar XMP (if any). The later call overwrites any field the earlier call set. Result: **sidecar wins over embedded**.

Rationale: the sidecar (if it still exists) is by definition older than Magpie's current write path, but on first scan after upgrade we must respect edits already stored there. The precedence rule becomes moot after the first save into the source file, because `write_metadata_to_source` deletes the sidecar as part of its cleanup step.

4. Technical fields folded in

The scanner reads `technical` entries whose keys match well-known labels (`Dimensions`, `Taken`, `Make`, `Model`) and folds them into `ImageMetaFromFile` for storage in the DB. The rest of the list is purely display-time; it isn't persisted.

XMP parser

`xmp_packet::parse_xmp(bytes) -> XmpUserMeta` is a hand-written streaming parser built on `quick_xml`. Uses a simple state machine and normalises casing/namespaces because tools in the wild are notoriously inconsistent (I've seen `<X:XMPMETA>`, `<x:xmpmeta>`, and mixed-case in the same file). See the `read_sidecar_case_variants` test.

Recognises both `dc:subject` (standard) and `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer); unions the two sets, deduping case-insensitively while preserving the first observed casing. Handles both `<rdf:Alt>` and `<rdf:Seq>` inside `<dc:title>`; the `x-default` language item wins if present, otherwise the first item.

FS-refresh check

Inside `get_image`, before returning, we call:

```
fn refresh_needed_from_fs(image_path: &Path, cached: &ImageDetails) -> bool {
    let last_read = cached.meta_read_at.unwrap_or(0);
    let src_mtime = mtime_ms(image_path);
    let sidecar_mtime = mtime_ms(&sidecar_path_for(image_path));
    src_mtime > last_read || sidecar_mtime > last_read
}
```

If it returns `true`, we call `read_all`, `resync_user_meta_from_fs` (which updates the DB row), and `set_meta_read_at_now` — then serve the fresh state.

We still watch the legacy sidecar's mtime here because a user might have run Lightroom against the same folder between two Magpie scans, updating a `.xmp` without touching the source. Once Magpie saves, the sidecar is cleaned up and the source-file mtime becomes the sole trigger.

This is how an external tag edit (Windows Explorer) becomes visible in Magpie on next click, without needing a full library rescan.

Metadata write path

Entry point

```
pub fn write_metadata_to_source(  
    registry: &FormatRegistry,  
    path: &Path,  
    patch_title: Option<Option<String>>,  
    patch_subjects: Option<Vec<String>>,  
) -> AppResult<>;
```

Called from `apply_patch_and_persist` in `commands/images.rs`, which is itself called by both `update_image_metadata` and `batch_update_metadata`.

Policy. All user metadata Magpie edits is embedded directly inside the source file. Magpie never creates `.xmp` sidecar files. The dispatcher picks one of two write paths:

1. **Native handler** — for formats where Magpie can rewrite the container itself (JPEG, PNG, WebP, GIF89a today).
2. **Windows Shell fallback** (`core::formats::win_shell`) — for every other registered format on Windows. This uses `IPropertyStore` (`SHGetPropertyStoreFromParsingName` + `SetValue` + `Commit`), the same COM interface behind Explorer's *Properties* → *Details* dialog. Anything a user could tag by hand in Explorer is writable by Magpie via this path, and vice-versa.

Format dispatch

```
let ext = path.extension().and_then(|s| s.to_str()).unwrap_or("");
let handler = registry.for_ext(ext).ok_or(AppError::UnsupportedFormat)?;

let existing = if handler.can_write_tags() {
    handler.read_user(path).unwrap_or_default()
} else {
    win_shell::read_user_meta(path).unwrap_or_default()
};
let edits = merge(existing, patch_title, patch_subjects);

if handler.can_write_tags() {
    handler.write_user(path, &edits)?;           // native
} else {
    win_shell::write_user_meta(path, &edits)?;    // Windows Shell
}
```

The registry decides which native handler owns a given extension. `can_write_tags()` on the handler decides whether we go native or fall back to the Shell. See [File formats](#) for the handler catalogue.

The IPC-facing `ImageDetails.writeMode` sent to the frontend is computed with the same priority the writer uses at dispatch time:

```
let write_mode = if native_can_write {
    WriteMode::Native           // JPEG / PNG / WebP / GIF89a
} else if services.shell_can_write_tags(path) {
    WriteMode::Shell           // RAW / HEIC / TIFF / MP4 / MOV / ...
} else {
    WriteMode::LibraryOnly      // BMP / DIB / SVG / EXR / HDR / ...
};
let can_write_tags =
    !matches!(write_mode, WriteMode::LibraryOnly);
```

`shell_can_write_tags` is a per-extension cache on top of `win_shell::can_write_tags(path)`, which performs a **two-stage** probe:

1. Open the store with `SHGetPropertyStoreFromParsingName(..., GPS_READWRITE, IPropertyStore)`. Extensions with no writable property handler (`STG_E_ACCESSDENIED` / `REGDB_E_CLASSNOTREG`) fail out here — SVG, EXR, HDR, PDF-without-Adobe, WebM, MKV, ...
2. Ask the store *per-property* whether it accepts writes to `System.Keywords` via `IPropertyStoreCapabilities::IsPropertyWritable(&PKEY_Keywords)`. This step is what filters out BMP, DIB, GIF-shell, and WebP-shell — their handlers *open* a R/W store but reject `SetValue` on Keywords. Without this check the UI would happily enable editing and then silently drop the user's tags.

The frontend uses `writeMode` (not just `canWriteTags`) to show one of three hint strings under the tag editor: “*embedded directly ... (XMP)*”, “*saved via the same Windows property system as Explorer’s Details tab*”, or “*stored in Magpie’s library only*”.

Handlers with native `can_write_tags() == true`:

Handler	Container	Notes
JPEG (<code>.jpg</code> , <code>.jpeg</code>)	APP1 XMP segment	Drops any existing standard-XMP or ExtendedXMP APP1 to avoid stacking.
PNG (<code>.png</code>)	iTXt chunk with keyword <code>XML:com.adobe.xmp</code>	Emitted immediately after IHDR; CRC recomputed.
WebP (<code>.webp</code>)	RIFF <code>XMP</code> chunk	Upgrades simple-form VP8L to extended VP8X on write if needed.
GIF89a (<code>.gif</code>)	Application Extension block with Adobe’s XMP magic trailer	Rejects GIF87a with a clear error.

Every other registered extension (RAW families, MP4/MOV, MKV, WMV, PDF, HEIC/AVIF, JPEG XL, TIFF/DNG, ...) is served by the Windows Shell fallback. Whether that fallback actually succeeds is a runtime property of the machine: if `SHGetPropertyStoreFromParsingName(..., GPS_READWRITE, IPropertyStore)` returns a live store for the file, Magpie writes through it; otherwise it surfaces the OS error verbatim wrapped in an `AppError::MetadataWrite`.

On non-Windows platforms `win_shell::write_user_meta` is a stub that always returns `Err(MetadataWrite("Windows Shell property system is not available on this platform."))` until an equivalent per-platform fallback ships.

Algorithm inside a writable handler

1. **Read the whole file.** All target formats top out well below typical RAM; a 20 MB JPEG is negligible.
2. **Extract the existing XMP** using the format-specific walker (`extract_xmp`).
3. **Parse it** with `xmp_packet::parse_xmp` into an `XmpUserMeta`.
4. **Merge the incoming UserMeta** with `xmp_packet::merge_user_edits`. This preserves any foreign fields (`xmp:Rating` , `dc:description` , GPS, etc.) so a Lightroom user never loses their star ratings just because Magpie doesn’t surface them.
5. **Rebuild the packet** with `xmp_packet::build_xmp_packet`.

6. **Rewrite the container** — dropping any prior XMP block and splicing the new one in at the spec-recommended location.
7. **Atomic write** via `common::atomic_write_bytes`.

Atomic file replace

Both writers funnel through `common::atomic_write_bytes`:

```
pub fn atomic_write_bytes(path: &Path, bytes: &[u8]) -> AppResult<()> {
    let tmp = path.with_file_name(
        format!("{}", path.file_name().unwrap().to_string_lossy(),
WRITE_TMP_SUFFIX)
    );
    { let mut f = File::create(&tmp)?; f.write_all(bytes)?; f.sync_all().ok(); }
    fs::rename(&tmp, path).or_else(|e| { let _ = fs::remove_file(&tmp); Err(e)
    })?;
    Ok(())
}
```

Windows semantics: `fs::rename` uses `MoveFileExW` internally with `MOVEFILE_REPLACE_EXISTING`, so the atomic swap works even when the target exists. On crash mid-write, the original file is still intact; at worst a temp file is left behind and is overwritten on the next successful write.

XMP packet builder

`xmp_packet::build_xmp_packet(&XmpUserMeta) -> String` emits a minimal, spec-conformant XMP packet:

```

<?xpacket begin="" id="W5M0MpCehiHzreSzNTczkc9d"?>
<x:xmpmeta xmlns:x="adobe:ns:meta/" x:xmptk="Magpie">
  <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
    <rdf:Description rdf:about=""
      xmlns:xmp="http://ns.adobe.com/xap/1.0/"
      xmlns:dc="http://purl.org/dc/elements/1.1/"
      xmp:Rating="4"
      xmp:MetadataDate="2026-08-17T22:38:11+03:00">

      <dc:title>
        <rdf:Alt>
          <rdf:li xml:lang="x-default">Sunset over Vík</rdf:li>
        </rdf:Alt>
      </dc:title>

      <dc:description>
        <rdf:Alt>
          <rdf:li xml:lang="x-default">Long exposure, ND1000.</rdf:li>
        </rdf:Alt>
      </dc:description>

      <dc:subject>
        <rdf:Bag>
          <rdf:li>Iceland</rdf:li>
          <rdf:li>sunset</rdf:li>
        </rdf:Bag>
      </dc:subject>

    </rdf:Description>
  </rdf:RDF>
</x:xmpmeta>
<?xpacket end="w"?>

```

Empty fields are omitted. Every text value is XML-escaped before insertion so ampersands, angle brackets, and quotes never break the packet. `xmp:Rating` and `dc:description` only appear if the file already had them — Magpie’s UI doesn’t produce those fields but does not clobber them.

Legacy sidecar cleanup

After a successful embed, if `sidecar_path_for(image_path)` exists, `write_metadata_to_source` removes it. The deletion is best-effort: failure is logged but doesn’t fail the save, because the source file already holds the authoritative data.

Failure modes and recovery

Failure	Effect
Unsupported extension (RAW, HEIC, TIFF, ...)	Handler's <code>write_user</code> returns <code>Err(_)</code> before any I/O.
DB transaction fails	Nothing is written to disk; caller receives <code>Err(_)</code> .
Source file read fails	Return <code>Err(_)</code> . DB update from earlier is kept (retryable).
Source file rewrite fails (tmp / rename)	Temp file removed if possible; original intact; caller sees <code>Err</code> .
Embed succeeds; sidecar delete fails	Save reports <code>ok(())</code> ; warning is logged; stale sidecar will be
	overridden by fresh embedded packet on next read.
Crash mid-write (any step)	Original file intact (atomic rename); temp file leaks and is
	eventually cleaned up on next successful write of same file.

Test coverage

Integration tests in `src-tauri/tests/metadata_fs.rs`:

- `read_sidecar_end_to_end` — legacy sidecar read.
- `read_sidecar_case_variants` — namespace/case robustness.
- `fts_delete_after_tag_update_works` — FTS5 regression.
- `batch_tag_add_persists_for_every_image` — batch semantics.
- `embed_xmp_roundtrip_{jpeg,png,webp,gif}` — one roundtrip per writable handler.
- `write_never_creates_sidecar_for_jpeg` — no `.xmp` fallback.
- `write_removes_legacy_sidecar_after_embed` — legacy sidecar cleanup.
- `write_errors_or_uses_shell_for_stub_format` — `.cr2` either writes through the Shell fallback or errors cleanly; never a sidecar.
- `write_preserves_foreign_rating_and_description` — rating and description written by other tools survive Magpie's edits.

- `registry_recognises_every_expected_extension` — every advertised handler is registered.

Unit tests in `core::formats::xmp_packet::tests`:

- `roundtrip_preserves_all_fields`
- `windows_explorer_tags_read`
- `microsoft_only_keywords_read`
- `merge_preserves_foreign_rating_and_description`

File formats

Magpie's tag/title write path is powered by a small trait-based plug-in system rather than a hard-coded `match ext { ... }`. This page describes what already exists.

The FormatHandler trait

Every supported file type is represented by a Rust type that implements the trait declared in `src-tauri/src/core/formats/mod.rs`:

```
pub trait FormatHandler: Send + Sync {
    fn name(&self) -> &'static str;
    fn extensions(&self) -> &'static [&'static str];
    fn kind(&self) -> FormatKind;
    fn can_write_tags(&self) -> bool;

    fn read_technical(&self, path: &Path) -> TechnicalMeta;
    fn read_user(&self, path: &Path) -> AppResult<UserMeta>;
    fn write_user(&self, path: &Path, meta: &UserMeta) -> AppResult<()>;
}
```

- `name` — human-readable label shown in the details panel (e.g. "JPEG (XMP APP1)").
- `extensions` — lowercase extensions this handler owns (`&["jpg", "jpeg"]`).
- `kind` — image / video / document / other. Used to classify entries and skip inappropriate operations (thumbnails, previews).
- `can_write_tags` — advisory. If `false`, the frontend disables the editable fields and shows a warning; the backend still returns a clean error from `write_user` if it's called anyway.
- `read_technical` — the read-only metadata shown at the bottom of the details panel. An ordered list of `(label, value)` pairs.
- `read_user` — the editable title / tags surface.
- `write_user` — atomically writes user meta into the source file.

The FormatRegistry

`FormatRegistry` maps each lowercase extension to exactly one handler. It's constructed once at startup, wrapped in `Arc`, and carried by `AppServices`.

Handlers register themselves in `FormatRegistry::new()` . The registry is the sole authority for:

- **Which extensions are scannable.** `scanner` calls `registry.for_ext(ext).is_some()` to decide whether to index a file.
- **Which extensions can be edited.** `meta_write::write_metadata_to_source` looks up the handler and calls `write_user` ; if the handler returns a `WriteNotSupported` error the write is refused cleanly (no fallback to sidecars).
- **Where technical/user metadata comes from.** `meta_read::read_all` builds an `ImageMetaFromFile` by calling `handler.read_technical(path)` and `handler.read_user(path)` , merged with any legacy sidecar values.

Writing tags: two paths

`FormatHandler::can_write_tags` reports only what the *native* handler can do. On Windows, `meta_write::write_metadata_to_source` also consults `core::formats::win_shell` , which wraps `SHGetPropertyStoreFromParsingName + IPropertyStore` — the same COM interface behind Explorer’s *Properties* → *Details* dialog.

Path	When it’s used	What it writes
Native handler	<code>handler.can_write_tags() == true</code>	The full XMP packet inside the container. Preserves foreign fields (<code>xmp:Rating</code> , <code>dc:description</code>).
Windows Shell fallback	Otherwise, on Windows	<code>System.Title</code> and <code>System.Keywords</code> via <code>IPropertyStore</code> . Any other properties (rating, author, GPS) are left as they are.

The IPC-facing `ImageDetails.canWriteTags` is the OR of the two, and its per-extension answer for the Shell fallback is cached on `AppServices::shell_write_cache` . On non-Windows platforms `win_shell` compiles to a no-op that reports “not available”.

Handler catalogue

The “Native writes?” column is what `FormatHandler::can_write_tags` returns. The “Shell fallback?” column is what a stock Windows 10 / 11 install typically offers via `IPropertyStore` ; it depends on which property handlers / codec packs the user has installed.

Handler	Kind	Extensions	Native writes?	Shell fallback?	Notes
JPEG	Image	.jpg , .jpeg	✓	not needed	XMP APP1 segment.
PNG	Image	.png	✓	not needed	iTXt chunk with XML:com.adobe.xmp .
WebP	Image	.webp	✓	not needed	XMP RIFF chunk. Converts simple-form VP8L to extended VP8X on write when needed.
GIF	Image	.gif	✓ (GIF89a only)	not needed	Adobe's XMP-in-GIF Application Extension.
TIFF	Image	.tif , .tiff , .dng	✗	✓	Reads tag 700 XMP + EXIF natively; write goes through the Shell.
HEIC / HEIF / AVIF	Image	.heic , .heif , .avif	✗	✓	Native read of box-level EXIF; Shell writes.
JPEG XL	Image	.jxl	✗	⚠ handler-dependent	Windows JXL codec pack ships the property handler; if not installed, library-only.
JPEG 2000	Image	.jp2 , .j2k , .jpx	✗	⚠	Same as JXL.
PSD	Image	.psd , .psb	✗	✓	Photoshop ships a property handler; Windows also ships one for basic keywords.
PDF	Document	.pdf	✗	⚠	Requires an installed PDF property handler

Handler	Kind	Extensions	Native writes?	Shell fallback?	Notes
					(Adobe Reader, PDF-XChange, ...).
MP4/MOV	Video	.mp4 , .m4v , .mov	✗	✓	Native duration/resolution read; Shell writes tags into moov/udta boxes.
Matroska / WebM	Video	.mkv , .webm	✗	⚠	Shell fallback works if the Matroska property handler is installed (VideoLAN / K-Lite).
AVI	Video	.avi	✗	✓	
WMV/ASF	Video	.wmv , .asf	✗	✓	
MPEG-TS	Video	.mpg , .mpeg , .mts , .m2ts	✗	⚠	Container-only native read.
3GP	Video	.3gp , .3g2	✗	✓	
RAW variants	Image	.cr2 , .cr3 , .nef , .arw , .raf , .orf , .rw2 , .pef , .srw , .x3f	✗	⚠ vendor codec pack	Native EXIF read; Shell writes when the vendor's Windows codec pack is present. Sigma X3F: Sigma Photo Pro / Windows RAW image extension.
BMP	Image	.bmp , .dib	✗	✓	Windows ships a property handler.
OpenEXR	Image	.exr	✗	✗	No standard Windows property handler ships for .exr ; library-only.

Handler	Kind	Extensions	Native writes?	Shell fallback?	Notes
Radiance HDR	Image	.hdr, .pic	✗	✗	Library-only.
SVG	Image	.svg, .svgz	✗	✗	Library-only.

Legend:  writes today,  never writes on the format,  depends on which property handler the user has installed on their machine.

Where the code lives

```
src-tauri/src/core/formats/
├── mod.rs                # FormatHandler trait + FormatRegistry + TechnicalMeta /
UserMeta
├── common.rs             # Shared atomic_write, EXIF → TechnicalMeta, dimensions
├── xmp_packet.rs         # XMP parse / build (single source of truth for XMP wire
format)
├── win_shell.rs          # IPropertyStore fallback (Windows only; stubbed
elsewhere)
├── jpeg.rs               # writable natively
├── png.rs                 # writable natively
├── webp.rs               # writable natively
├── gif.rs                 # writable natively (GIF89a only)
├── tiff.rs               # native read, Shell write
└── stubs.rs              # all the other read-natively handlers, one struct per
family
```

`stubs.rs` deliberately groups similar formats so adding a new “native read, Shell write” format is one struct + one line in `FormatRegistry::new`.

Adding a format handler

Adding support for a new file type is a five-step recipe. This page walks through it using a fictitious “MyFormat” (`.myf`) as an example.

1. Pick the right home

- **Read-only** (Magpie only extracts technical metadata): add a new struct to `src-tauri/src/core/formats/stubs.rs` — this file already hosts a couple of dozen similar handlers.
- **Read + write** (Magpie can embed tags into the file): create a new module `src-tauri/src/core/formats/myformat.rs`.

Both flavours implement the same trait; the only difference is that a writable handler contains real embedding code in `write_user`, while a read-only one returns `common::write_not_supported_error(...)`.

2. Implement FormatHandler

```
use super::common::{self, write_not_supported_error, TechnicalMeta};
use super::{FormatHandler, FormatKind, UserMeta};
use crate::error::AppResult;
use std::path::Path;

pub struct MyFormat;

impl FormatHandler for MyFormat {
    fn name(&self) -> &'static str {
        "MyFormat (read-only)"
    }
    fn extensions(&self) -> &'static [&'static str] {
        &["myf"]
    }
    fn kind(&self) -> FormatKind {
        FormatKind::Image
    }
    fn can_write_tags(&self) -> bool {
        false
    }

    fn read_technical(&self, path: &Path) -> TechnicalMeta {
        let mut t = TechnicalMeta::default();
        common::append_file_basics(&mut t, path);
        // ... push more pairs specific to this format ...
        t
    }

    fn read_user(&self, path: &Path) -> AppResult<UserMeta> {
        let _ = path;
        Ok(UserMeta::default())
    }

    fn write_user(&self, path: &Path, _meta: &UserMeta) -> AppResult<()> {
        write_not_supported_error(path, "myf")
    }
}
```

For a **writable** handler, `write_user` typically:

1. Reads the current bytes via `std::fs::read`.
2. Uses `xmp_packet::parse_xmp` to recover any existing XMP.
3. Merges the incoming `UserMeta` into that packet with `xmp_packet::merge_user_edits`, preserving foreign fields (rating, description, ...).
4. Rebuilds the packet with `xmp_packet::build_xmp_packet`.
5. Injects the new packet into the file's format-specific container.

6. Writes the result via `common::atomic_write_bytes` — never in-place.
`atomic_write_bytes` uses the tmp-file + rename pattern so a crash mid-write cannot corrupt the source.

3. Register it

Open `src-tauri/src/core/formats/mod.rs` and add the handler to `FormatRegistry::new:`

```
registry.register(Arc::new(myformat::MyFormat));
```

For a read-only stub, add it to `stubs::register` (already called from `FormatRegistry::new`).

4. Write tests

Add a roundtrip test in `src-tauri/tests/metadata_fs.rs`:

```
#[test]
fn embed_xmp_roundtrip_myformat() {
    let tmp = tempdir();
    let img = tmp.join("real.myf");
    std::fs::write(&img, tiny_myformat()).unwrap();
    let reg = registry();
    let h = reg.for_ext("myf").unwrap();
    h.write_user(&img, &UserMeta {
        title: Some("Sample".into()),
        tags: vec!["one".into(), "two".into()],
    }).unwrap();
    let after = h.read_user(&img).unwrap();
    assert_eq!(after.title.as_deref(), Some("Sample"));
    assert_eq!(after.tags, vec!["one".to_string(), "two".to_string()]);
}
```

For a read-only handler, add it to `registry_recognises_every_expected_extension` to prove the extension is scannable, and test that `write_user` returns an error that names the format.

5. Update the docs

- Add a row to the handler catalogue on [File formats](#).
- Add the extension to the user-manual [Supported file formats](#) page.

Design invariants

While writing a new handler, keep these invariants in mind:

- **Never create sidecar files.** If you can't embed, return an error via `write_not_supported_error`. The frontend gracefully handles that and Magpie remembers the tag in its own library instead.
- **Never write in-place.** Always go through `common::atomic_write_bytes` so the source file can never be left half-written.
- **Preserve foreign fields.** When rewriting an XMP packet, use the `merge_user_edits` helper — direct `title / tags` writes must not touch `dc:description`, `xmp:Rating`, GPS, or any other tags Magpie doesn't own.
- **Return the extension in error messages.** Callers surface these to the UI. Include the offending file extension for a good user message (see `write_not_supported_error`).
- **No blocking I/O on hot paths.** Handlers may be invoked from the scanner (parallel) and the metadata write path (foreground); restrict work in `read_technical` to what's cheap enough to run during a scan.

Testing strategy

Layers

L4: Manual smoke – screenshot scripts + real UI clicks	← rare
L3: Integration tests (Rust) src-tauri/tests/metadata_fs.rs	← ~5s per run
L2: Unit tests (Rust, inline <code>#[cfg(test)]</code>) xmp.rs (parse/build/CRC), migrations.rs	← <1s
L1: Type checks (TypeScript strict + cargo check)	← seconds

L1 — type checks

- `cargo check --workspace` for the Rust side (no unused warnings allowed in CI).
- `tsc -b --pretty false` for the frontend; strict mode enabled in `tsconfig.json` (`noUncheckedIndexedAccess`, `noImplicitAny`, `strictNullChecks`).

Both are cheap and run before every commit.

L2 — Rust unit tests

Colocated with modules in `#[cfg(test)] mod tests { ... }`. Examples:

- `core/metadata/xmp.rs` — parser tests for Windows Explorer tags, Microsoft-only keyword lists, case variants, attribute-form values. Also covers the JPEG APP1 walker and (in integration tests) the PNG iTXt CRC.
- `db/queries.rs` — smoke tests for FTS row rebuild.

Run with:

```
cd src-tauri
cargo test --lib
```

L3 — Rust integration tests

`src-tauri/tests/metadata_fs.rs` exercises the full pipeline on temporary directories:

Test	Verifies
<code>read_sidecar_end_to_end</code>	Legacy sidecar read: title + tags.
<code>read_sidecar_case_variants</code>	XMP parser is namespace/case insensitive.
<code>fts_delete_after_tag_update_works</code>	The FTS5 <code>contentless_delete</code> regression is fixed.
<code>batch_tag_add_persists_for_every_image</code>	Bulk <code>tags_add</code> / <code>tags_remove</code> semantics.
<code>embed_xmp_roundtrip_jpeg</code>	JPEG embed writes, re-reads, replaces on second write.
<code>embed_xmp_roundtrip_png</code>	PNG iTXt embed, no chunk stacking on rewrite.
<code>embed_xmp_roundtrip_webp</code>	WebP RIFF <code>xmp</code> chunk roundtrip, no stacking.
<code>embed_xmp_roundtrip_gif</code>	GIF89a Application Extension XMP roundtrip.
<code>write_never_creates_sidecar_for_jpeg</code>	Saving on a JPEG creates zero <code>.xmp</code> files.
<code>write_removes_legacy_sidecar_after_embed</code>	A pre-existing <code>.xmp</code> is removed on successful embed.
<code>write_preserves_foreign_rating_and_description</code>	Foreign <code>xmp:Rating</code> and <code>dc:description</code> survive a tag edit.
<code>write_errors_on_unsupported_format</code>	Saving on RAW returns <code>Err</code> , no sidecar fallback.

Test	Verifies
<code>registry_recognises_every_expected_extension</code>	Every advertised handler is registered.

Run:

```
cd src-tauri
cargo test --test metadata_fs
```

L4 — screenshot smoke tests

Under `scripts/`:

- `screenshot-app.ps1` — launches Magpie, finds its window with `EnumWindows`, screenshots via `PrintWindow ( PW_RENDERFULLCONTENT )` so the shot works even if another window is in front.
- `screenshot-scrolled.ps1` — clicks an image, scrolls the details panel, screenshots.
- `test-multiselect-tag.ps1` — end-to-end skeleton for Ctrl+click → type tag → click Apply. UI automation is inherently flaky on Windows; use these mostly to verify a build launches and paints, not for asserting behaviour.

These are not part of automated CI. They're one-off tools when investigating a UI-only bug.

Frontend tests

Currently minimal:

- Type safety is the main safety net.
- Component tests would use Vitest + React Testing Library; not set up in v1.

Because the UI is a thin translation over Rust commands, the interesting behaviour is on the Rust side and covered by L2/L3.

Format sample files

`samples/format-samples/` holds one average-sized synthetic file per handler registered in `FormatRegistry` (35 handlers, plus extension aliases such as `.jpeg`, `.tif`, `.m4v`).

Generate or refresh:

```
py -m pip install pillow reportlab pillow-heif imageio-ffmpeg
py samples/format-samples/generate_all.py
```

Output lands in `samples/format-samples/files/<ext>/sample.<ext>` with sizes and placeholder notes recorded in `manifest.json`. See `samples/format-samples/README.md` for caveats (camera RAW and some exotic codecs are scan-only placeholders, not vendor-native files).

Use these when manually exercising the scanner, thumbnail pipeline, or details panel against a broad extension set without copying a real library onto the machine.

Diagnostic tools (not tests, but adjacent)

- `src-tauri/examples/dump_meta.rs` — prints filesystem state, embedded XMP, any legacy sidecar contents, parsed metadata, and DB state for one photo or the first N rows in the DB. Great for reproducing bugs.
- `src-tauri/examples/dump_tag_usage.rs` — given a tag name, list every photo carrying it plus what the on-disk embedded XMP shows. Used in the “did the batch save actually work?” investigation.

Run:

```
cd src-tauri
cargo run --example dump_meta -- "C:\Photos\IMG_1234.jpg"
```

CI (recommended)

Not shipped in the repo yet, but the natural setup is:

- Windows runner (matrix: `windows-2022`).
- Steps: `cargo fmt --check`, `cargo clippy --all-targets`, `cargo test --workspace`, `npm ci`, `tsc -b`, `npm run tauri build -- --no-bundle`.

- Artifacts: `desktop.exe` for smoke inspection.

Build and release

Prerequisites (Windows)

```
# Rust toolchain (user scope)
winget install --id Rustlang.Rustup --accept-source-agreements

# MSVC C++ Build Tools (machine scope; needs UAC)
winget install --id Microsoft.VisualStudio.2022.BuildTools `
  --override "--wait --passive --add Microsoft.VisualStudio.Workload.VCTools --
  includeRecommended"

# Node LTS
winget install --id OpenJS.NodeJS.LTS
```

Verify:

```
rustc --version    # ≥ 1.77.2
cargo --version
node --version     # ≥ 18
npm --version
```

Clone and build

```
git clone <repo> magpie
cd magpie
npm install
```

Development

```
npm run tauri dev
```

This starts:

- **Vite dev server** on `localhost:1420` with HMR.
- **Cargo watch** compiling the Rust core.
- **Tauri window** loading the dev server URL.

Rust code changes trigger a full app rebuild (10–30 s); frontend changes hot-reload in <1 s.

Release build

```
npm run tauri build -- --no-bundle
```

- `--no-bundle` skips the MSI/NSIS installer, producing just `src-tauri/target/release/desktop.exe` . Faster and useful for local testing.
- Remove `--no-bundle` (or pass `--bundles msi` / `--bundles nsis`) to produce an installer under `src-tauri/target/release/bundle/` .

Timings on a modern laptop:

- Cold release build: ~5 min.
- Incremental release build after a small Rust change: ~90 s.
- Incremental after frontend change only: ~30 s.

Tauri configuration

Key fields in `src-tauri/tauri.conf.json` :

Field	Notes
<code>productName</code>	<code>Magpie</code>
<code>identifier</code>	<code>com.magpie.app</code> — used for <code>%APPDATA%</code> folder name.
<code>security.csp</code>	Strict; blocks inline scripts and remote origins.
<code>app.withGlobalTauri</code>	<code>false</code> — commands accessed via <code>@tauri-apps/api</code> only.
<code>bundle.icon</code>	Multi-size Windows ICO.
<code>bundle.windows.wix</code>	(Present in future for signed MSI.)

Capabilities (`src-tauri/capabilities/default.json`) declare only what's needed: dialog and opener plugins, asset access under the user's library folders. Network is *not* granted.

Release profile

src-tauri/Cargo.toml:

```
[profile.release]
codegen-units = 16
lto           = false
opt-level     = 3
panic         = "abort"
strip         = true
incremental   = false
```

Rationale:

- `codegen-units = 16` + `lto = false` — trades a bit of runtime perf for much faster incremental builds. In practice, the hot paths are limited by `image` decode and disk I/O, not by cross-crate inlining.
- `panic = "abort"` — smaller binary, no unwinding tables. All panics are treated as bugs; the app crashes and restarts.
- `strip = true` — removes debug symbols to keep the binary small (~13 MB stripped vs. 45 MB with symbols).

Version bump

1. `Cargo.toml`: `version = "0.1.1"` (both files if we ever add a workspace).
2. `package.json`: same.
3. `tauri.conf.json`: `version = "0.1.1"`.
4. Commit as `chore: v0.1.1`.

Releasing

The v1 workflow is manual:

1. `npm run tauri build` (with default bundle) on a clean tree.
2. Test the resulting MSI on a clean VM.
3. Draft a GitHub release with the MSI attached.
4. Attach the two PDFs from `docs/` as documentation assets.
5. Tag the commit `v0.1.0` and publish.

A GitHub Actions workflow is not shipped in v1 but can plug into the recommended CI setup in [Testing strategy](#).